

TCP

Control de Flujo

Control de Congestión

Redes (TA048)

Universidad de Buenos Aires, Facultad de Ingeniería



¿Qué características tiene TCP?

¿Qué nos ofrece?

¿Ventajas y Desventajas?



This is a TCP joke

Do you get it?
Do you get it?
Do you get it?

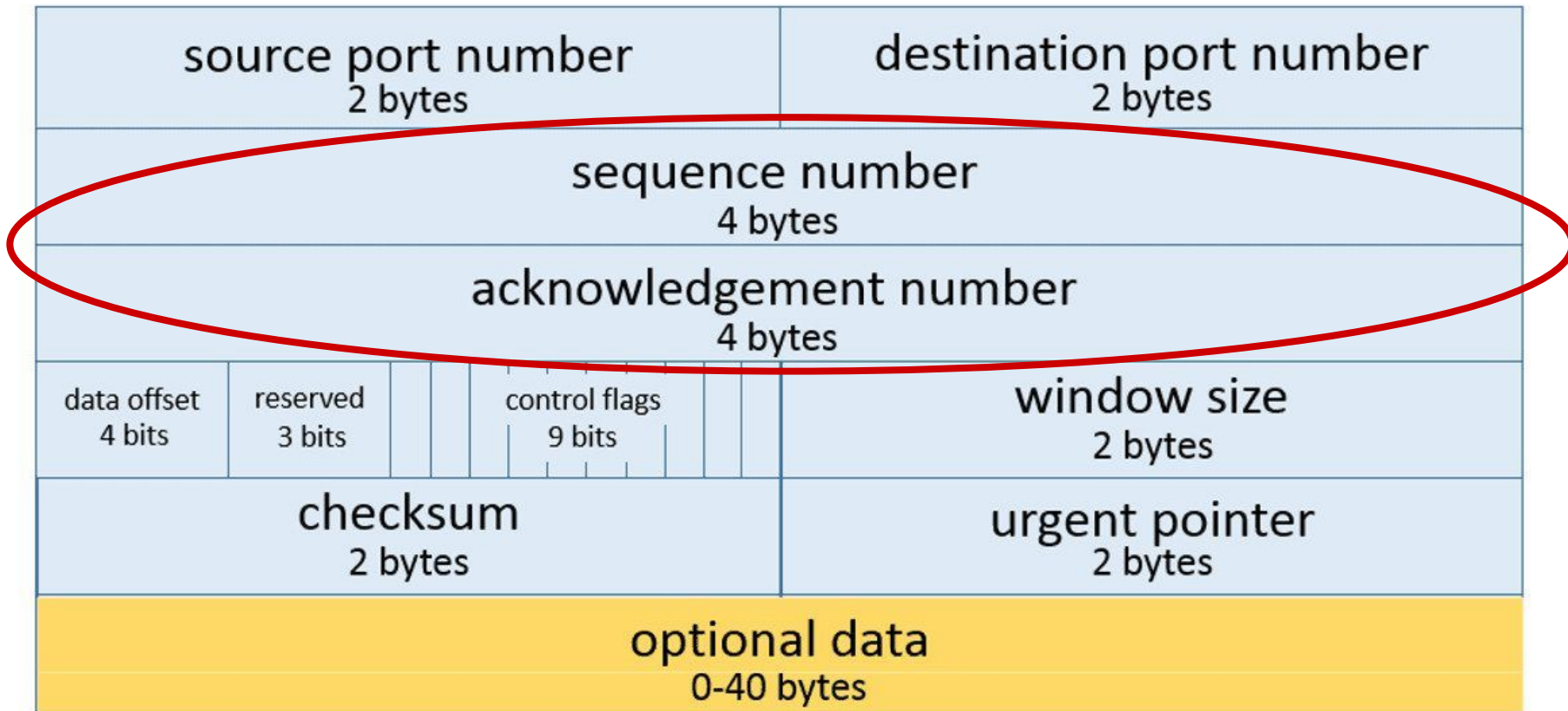
Transmission Control Protocol (TCP) Header

20-60 bytes

source port number 2 bytes				destination port number 2 bytes			
sequence number 4 bytes							
acknowledgement number 4 bytes							
data offset 4 bits		reserved 3 bits		control flags 9 bits		window size 2 bytes	
checksum 2 bytes				urgent pointer 2 bytes			
optional data 0-40 bytes							

TCP: #Sequence y #ACK

Transmission Control Protocol (TCP) Header 20-60 bytes



Control de flujo

Control de flujo

- Cada extremo de la transmisión TCP tiene su propio buffer.
- TCP almacena datos en el buffer para pasarlos en orden a la capa de aplicación.
- El buffer tiene un tamaño finito.



Control de flujo

¿Qué pasa si se reciben más datos de los que la capa de aplicación es capaz de consumir?

Control de flujo

¿Qué pasa si se reciben más datos de los que la capa de aplicación es capaz de consumir?

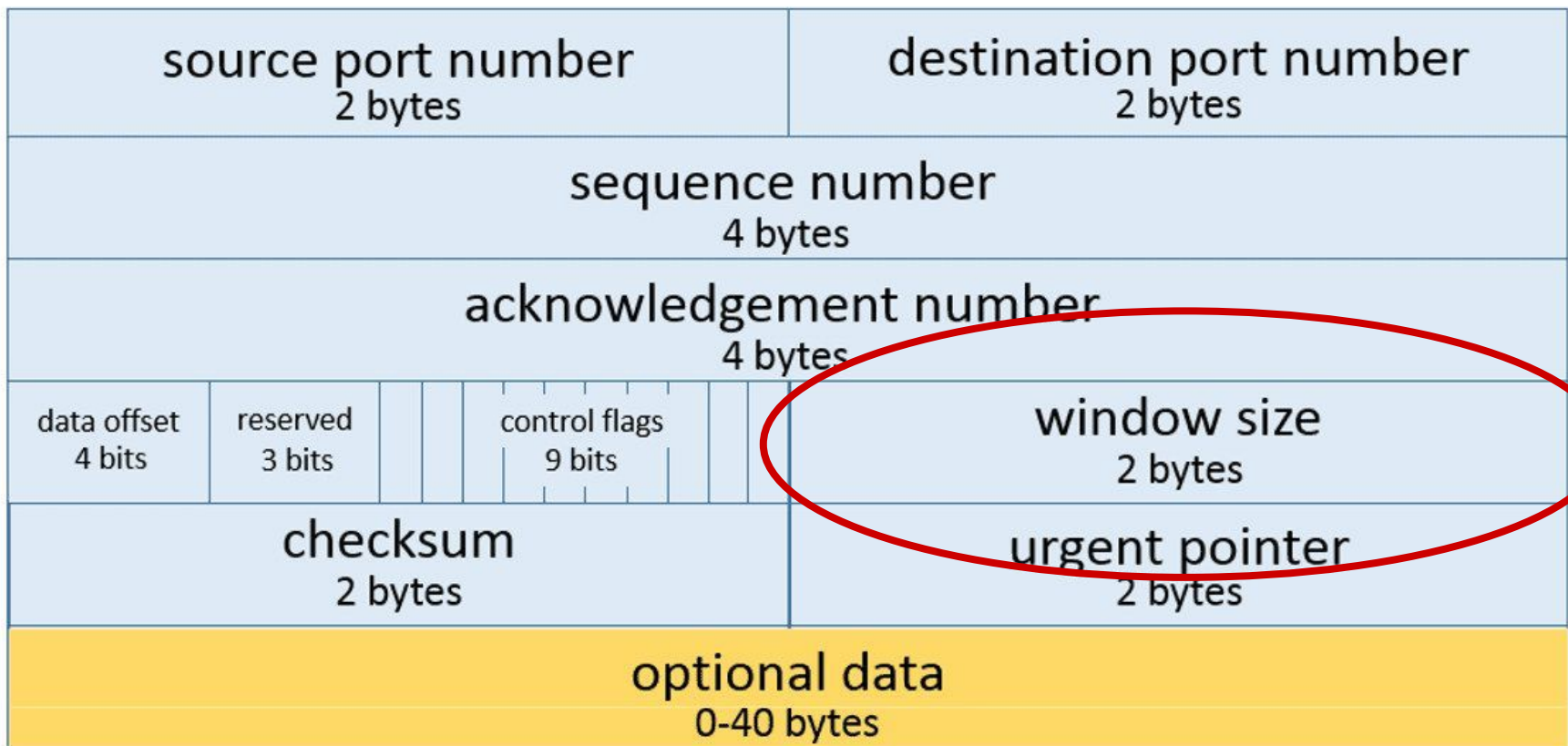
BUFFER OVERFLOW

Control de flujo

- Para evitar el buffer overflow, **cada extremo** de la transmisión tiene que comunicar la cantidad de datos que puede recibir.
- A esto se lo llama ***rwnd*** (Receiver Window o Advertised Window).
- Se utiliza el campo Window del TCP header para comunicar este valor.

TCP: Advertised Windows

Transmission Control Protocol (TCP) Header 20-60 bytes



Control de flujo

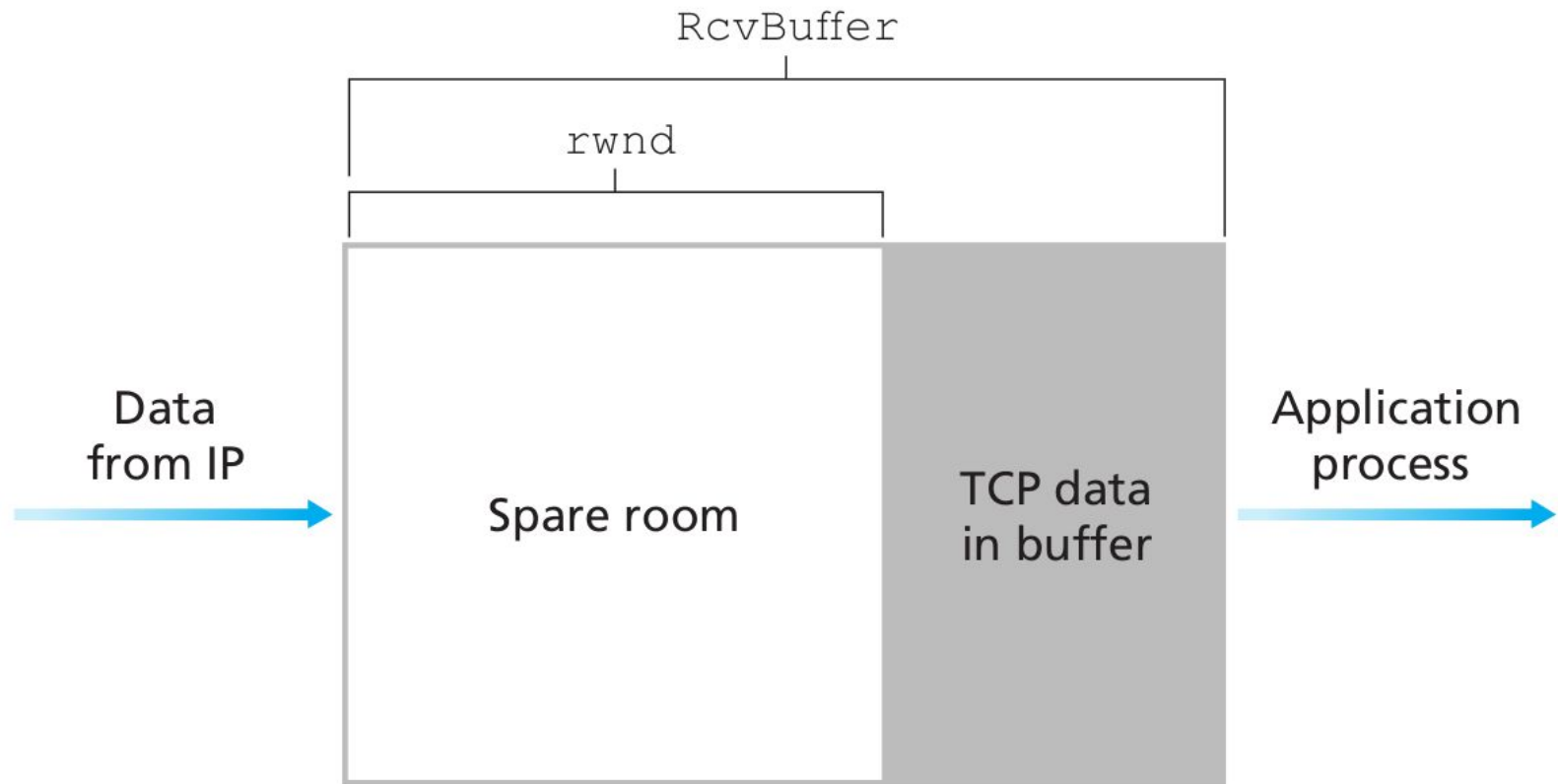


Figure 3.38 ♦ The receive window (**rwnd**) and the receive buffer (**RcvBuffer**)

Control de flujo

- Cuando se recibe un cambio en ***rwnd***, el emisor debe modificar su tasa de envío para no causar buffer overflow, pero tampoco subutilizar el buffer.
- Si se recibe ***rwnd*** = 0, el emisor debe dejar de enviar datos hasta recibir un nuevo valor de ***rwnd***

Control de flujo

- Imaginemos el siguiente escenario
 - A envía un paquete a B
 - El buffer de B se llena, por lo que envía un **ACK *rwnd* = 0** a A.
 - A deja de enviar datos.



Control de flujo

- La capa de aplicación en B lee del buffer y lo libera.

¿Cómo hace B para avisarle a A que puede recibir más datos?



Control de flujo

- Para lograr esto, luego de un período de inactividad (***persist timer***) A enviará paquetes de 1 byte (***window probe***) a B.
- Cuando B los recibe, indica el tamaño de ventana actual.
- Cuando el buffer se libere, B indicará ***rwnd*** > 0 y se reanudará la transmisión.



Control de congestión

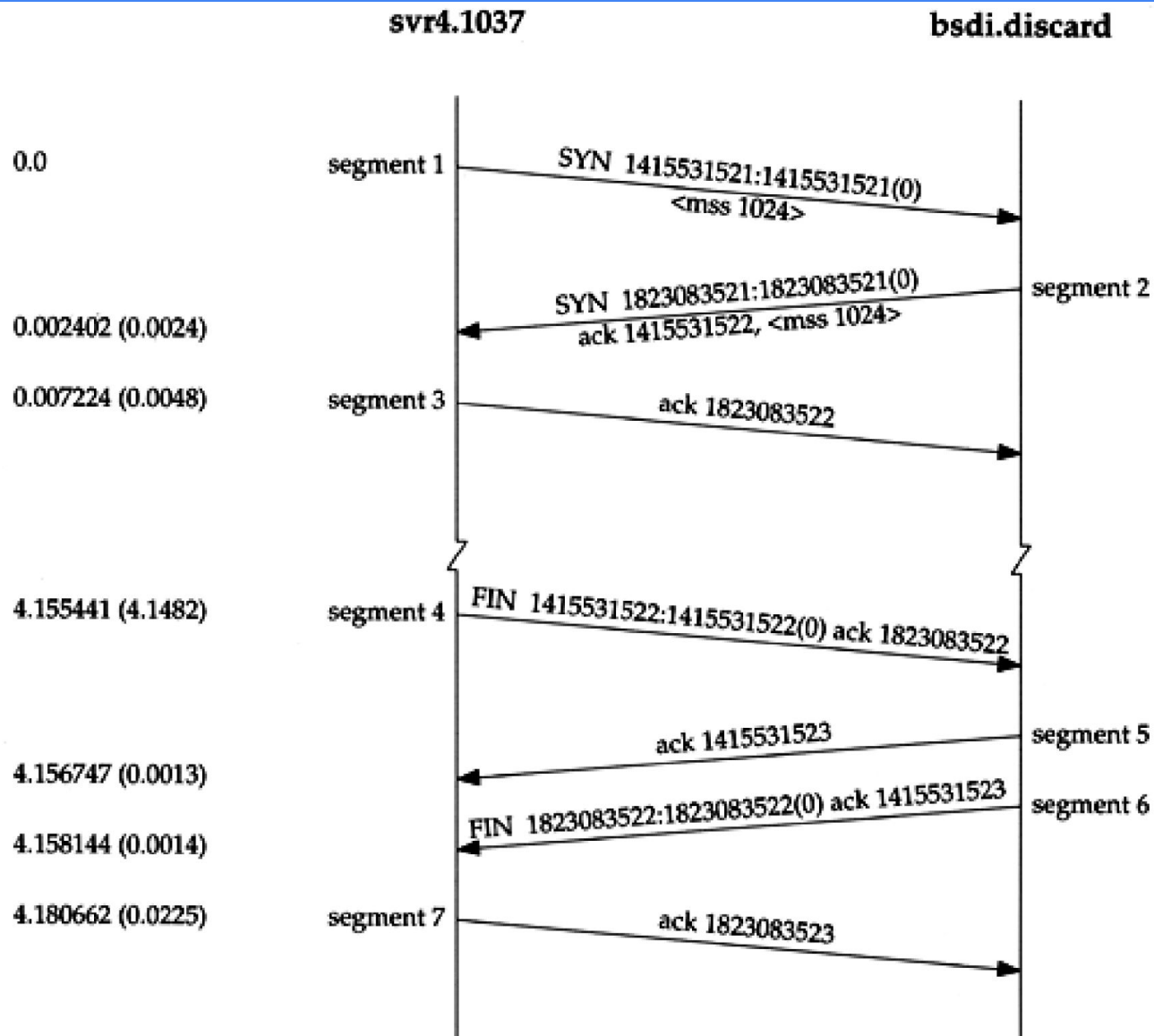
TCP : control de congestión

- El control de congestión **no formaba parte** del diseño original de TCP

TCP : control de congestión

- Fue una respuesta a la **congestión persistente** observada en la red
- La **congestión** es producto de múltiples flujos TCP en simultáneo sobre la misma red

TCP : control de congestión



TCP : control de congestión

Disclaimer: Modelo aproximado a la realidad utilizado para entender el funcionamiento de estos algoritmos.

Algoritmos de control de congestión



TCP : control de congestión

- La congestión aparece principalmente a causa del **tamaño finito** de los buffers de los **dispositivos intermedios** de capas inferiores.
- Cuando se llena un buffer, se empiezan a **descartar** los segmentos -> se dice que la red está **congestionada**

TCP : control de congestión

- **cwnd** : es la máxima cantidad de bytes que puede haber en vuelo
- **IW**: *initial window* es el valor inicial de cwnd. Tiene que ser ≤ 2 segmentos.

RECORDAR:

$\text{LastByteSent} - \text{LastByteACKed} < \min(\text{cwnd}, \text{rwnd})$

TCP : control de congestión

- Tiene 3 etapas distintas:
 - Slow start
 - Congestion avoidance
 - Fast Recovery
- Más el mecanismo Fast Retransmit

TCP : control de congestión

Slow start (real)

- $\text{cwnd}(n+1) = \text{cwnd}(n) + \text{MSS} * \#(\text{ACK})$
- cwnd medido en Bytes
- Crecimiento exponencial

TCP : control de congestión

Slow start (simplificado p/ejerc.)

- $\text{cwnd}(n+1) = \text{cwnd}(n) + \#(\text{ACK})$
- cwnd medido en **MSSs** (segmentos de tamaño máximo)
- Crecimiento exponencial

TCP : control de congestión

Slow start

$$cwnd(n+1) = cwnd(n) + \#(ACK)$$

- Ejemplo: si $cwnd(0) = 1$
- $cwnd(0) = 1$, envió 1 segmento

TCP : control de congestión

Slow start

$$cwnd(n+1) = cwnd(n) + \#(ACK)$$

- Ejemplo: si $cwnd(0) = 1$
- $cwnd(0) = 1$, envió 1 segmento
- Recibo 1 ACK
- $cwnd(1) = ?$

TCP : control de congestión

Slow start

$$cwnd(n+1) = cwnd(n) + \#(ACK)$$

- Ejemplo: si $cwnd(0) = 1$
- $cwnd(0) = 1$, envió 1 segmento
- Recibo 1 ACK
- $cwnd(1) = 1 + \#(ACK) = 1 + 1 = 2$

TCP : control de congestión

Slow start

$$cwnd(n+1) = cwnd(n) + \#(ACK)$$

- Ejemplo: si $cwnd(0) = 1$
- $cwnd(0) = 1$, envío 1 segmento
- Recibo 1 ACK
- $cwnd(1) = 1 + \#(ACK) = 1 + 1 = 2$
- Envío 2 segmentos. Recibo 2 ACK
- $cwnd(2) = ?$

TCP : control de congestión

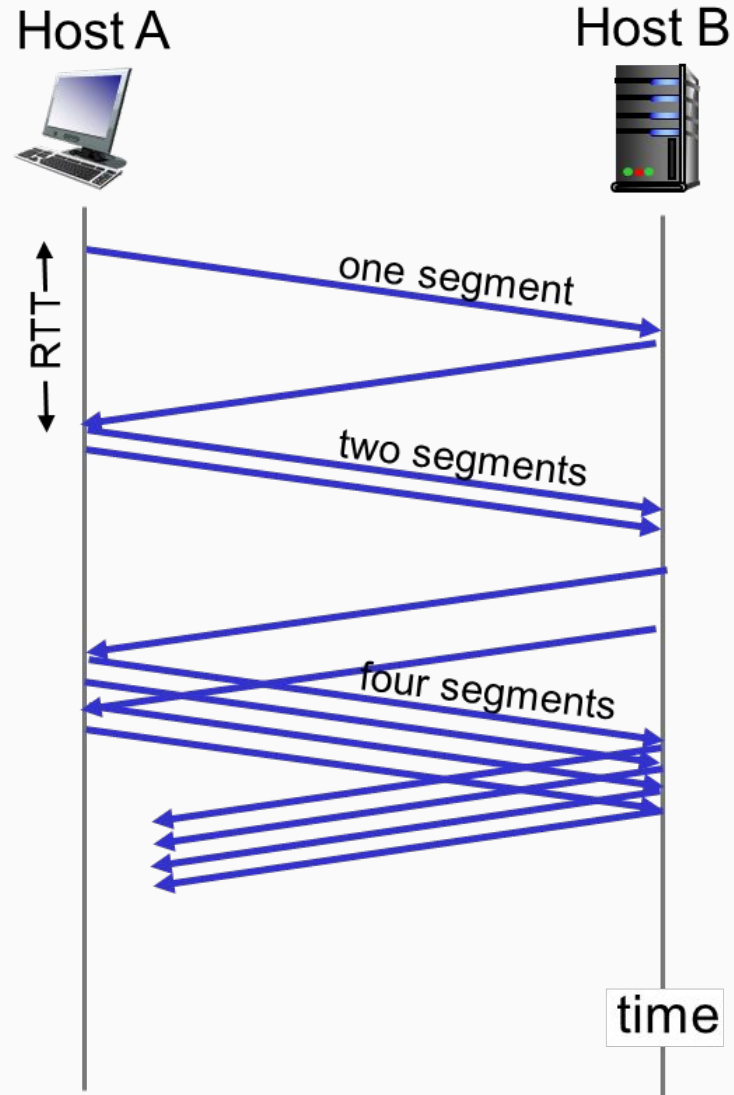
Slow start

$$cwnd(n+1) = cwnd(n) + \#(ACK)$$

- Ejemplo: si $cwnd(0) = 1$
- $cwnd(0) = 1$, envío 1 segmento
- Recibo 1 ACK
- $cwnd(1) = 1 + \#(ACK) = 1 + 1 = 2$
- Envío 2 segmentos. Recibo 2 ACK
- $cwnd(2) = 2 + \#(ACK) = 2 + 2 = 4$

TCP : control de congestión

Slow start

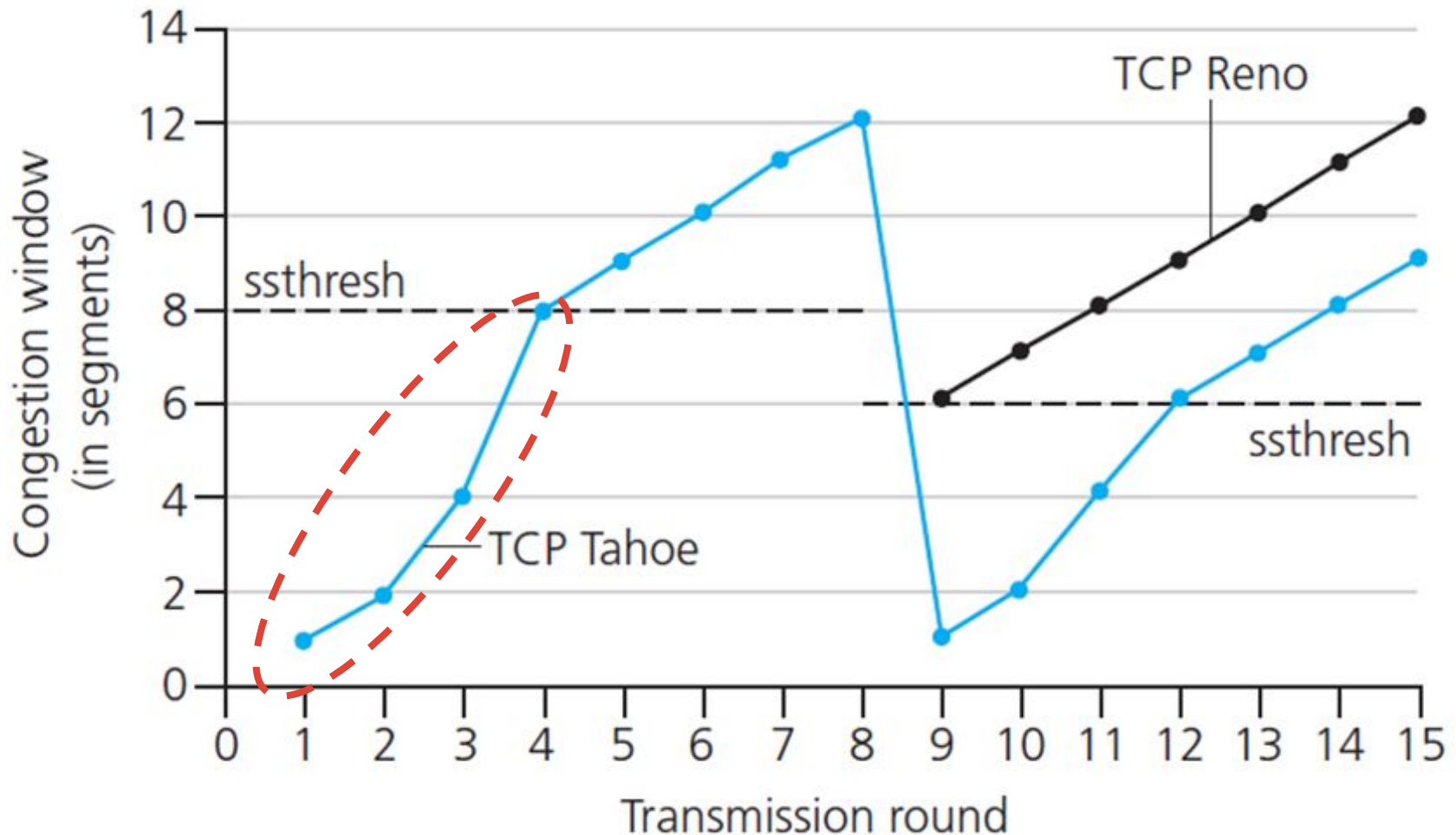


TCP : control de congestión

¿Seguimos
incrementando la ventana
exponencialmente para
siempre?

TCP : control de congestión

Slow Start



TCP : control de congestión

Cambio de etapa

- En un determinado momento, se dará por terminada la etapa de **Slow Start** para pasar a la próxima, llamada **Congestion Avoidance**
- Esto viene dado por el valor de **sstresh** (slow start threshold size), qué es configurable.

TCP : control de congestión

Congestion avoidance

- $\text{cwnd}(n+1) = \text{cwnd}(n) + \#(\text{ACK})/\text{cwnd}(n)$
- Aumenta 1 cuando nos llegan los ACKs de toda la ráfaga (1 MSS por RTT)
- cwnd siempre tiene que ser entero (la unidad es el **MSS**)
- Se redondea para abajo!

TCP : control de congestión

Congestion avoidance

$$cwnd(n+1) = cwnd(n) + \#(ACK)/cwnd(n)$$

- Ejemplo: si $cwnd(n) = 2$
- $cwnd(n) = 2$, envió 2 segmentos

TCP : control de congestión

Congestion avoidance

$$cwnd(n+1) = cwnd(n) + \#(ACK)/cwnd(n)$$

- Ejemplo: si $cwnd(n) = 2$
- $cwnd(n) = 2$, envió 2 segmentos
- Llega 1 ACK
- $cwnd(n+1) = 2 + 1 / 2 = 2$

TCP : control de congestión

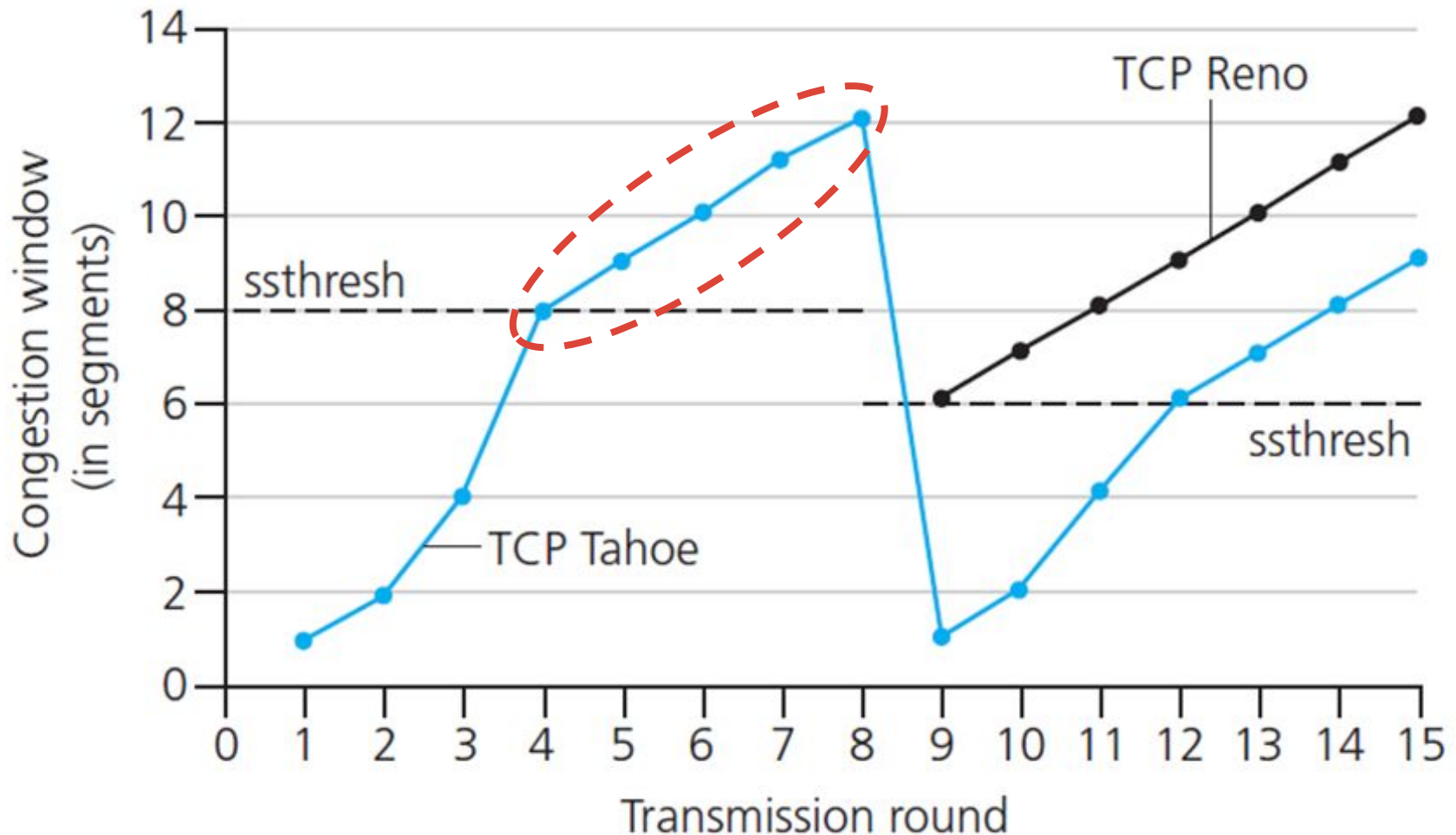
Congestion avoidance

$$cwnd(n+1) = cwnd(n) + \#(ACK)/cwnd(n)$$

- Ejemplo: si $cwnd(n) = 2$
- $cwnd(n) = 2$, envió 2 segmentos
- Llega 1 ACK
- $cwnd(n+1) = 2 + 1 / 2 = 2$
- Llega el 2do ACK
- $cwnd(n+2) = 2 + 2 / 2 = 3$

TCP : control de congestión

Congestion avoidance



TCP : control de congestión

¿Seguimos en CA para siempre, felizmente incrementando en $\#(\text{ACK})/\text{cwnd}(n)$ por toda la eternidad?

TCP : control de congestión

¿Qué pasa si se pierden
paquetes?

TCP: Perdidas por Timeout

Control de Congestión: Timeout

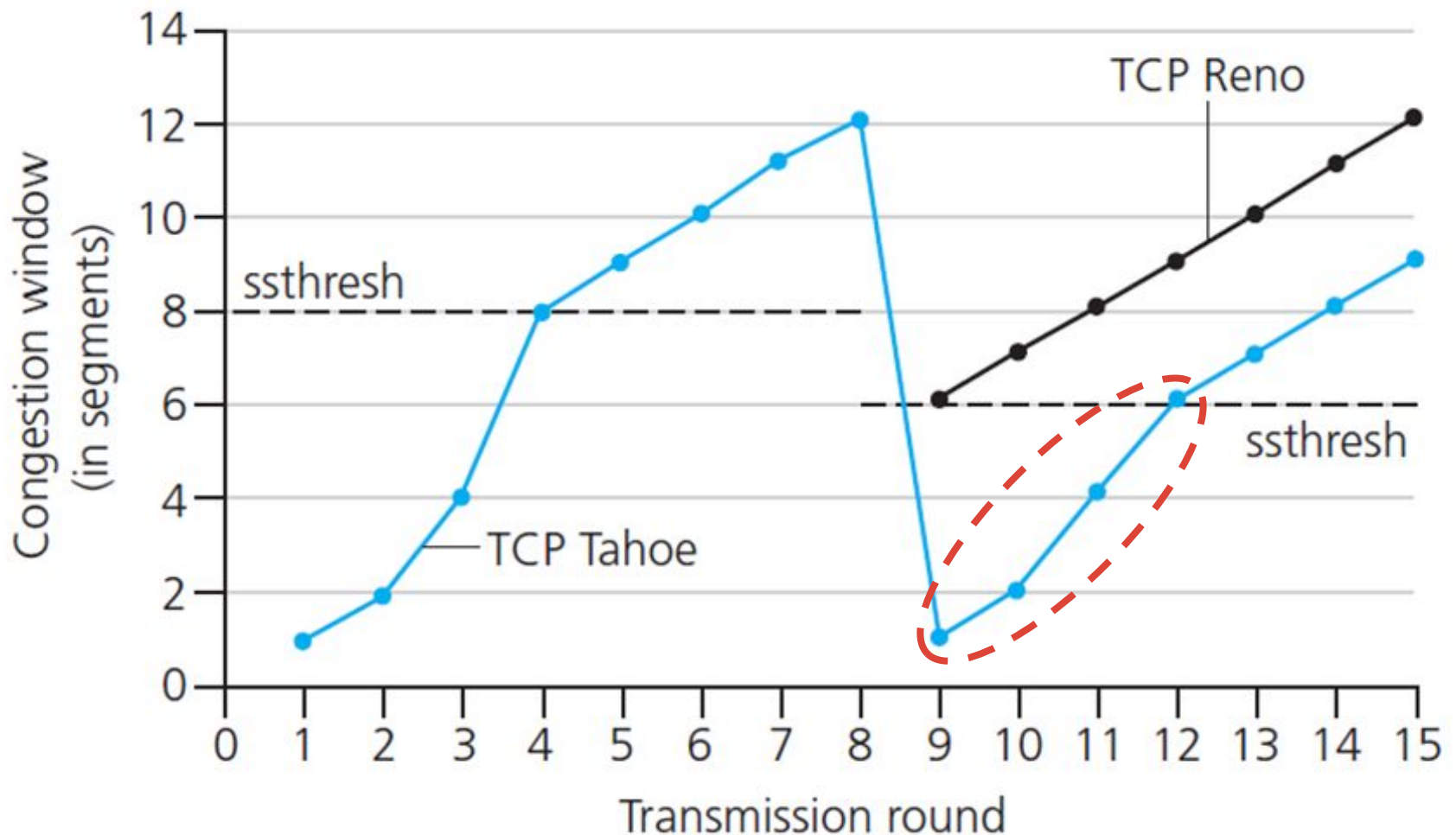
Pérdida por timeout (RTO)

- $ssthresh = cwnd(n) / 2$
- $cwnd(n+1) = 1$ (1 es LW [loss window])
- Empieza slow start de nuevo

Las pérdidas por Timeout aparecen cuando la congestión es grande (no llega ningún ACK)

Control de Congestión: Timeout

Timeout => Slow Start de nuevo



Pérdida de paquetes

- ¿Qué pasaría si se pierde un paquete?
- Supongamos que se pierde **UN SOLO** segmento de una ráfaga.

TCP : control de congestión

ACK duplicados (llegan algunos paquetes)

- Ocurre cuando se reciben 4 ACKs para el mismo segmento (3 dupACKs)
- Se inicia **Fast Retransmit**
- Depende del algoritmo implementado
 - **Fast Recovery**
 - **Slow Start**

ACKs Duplicados

Tahoe

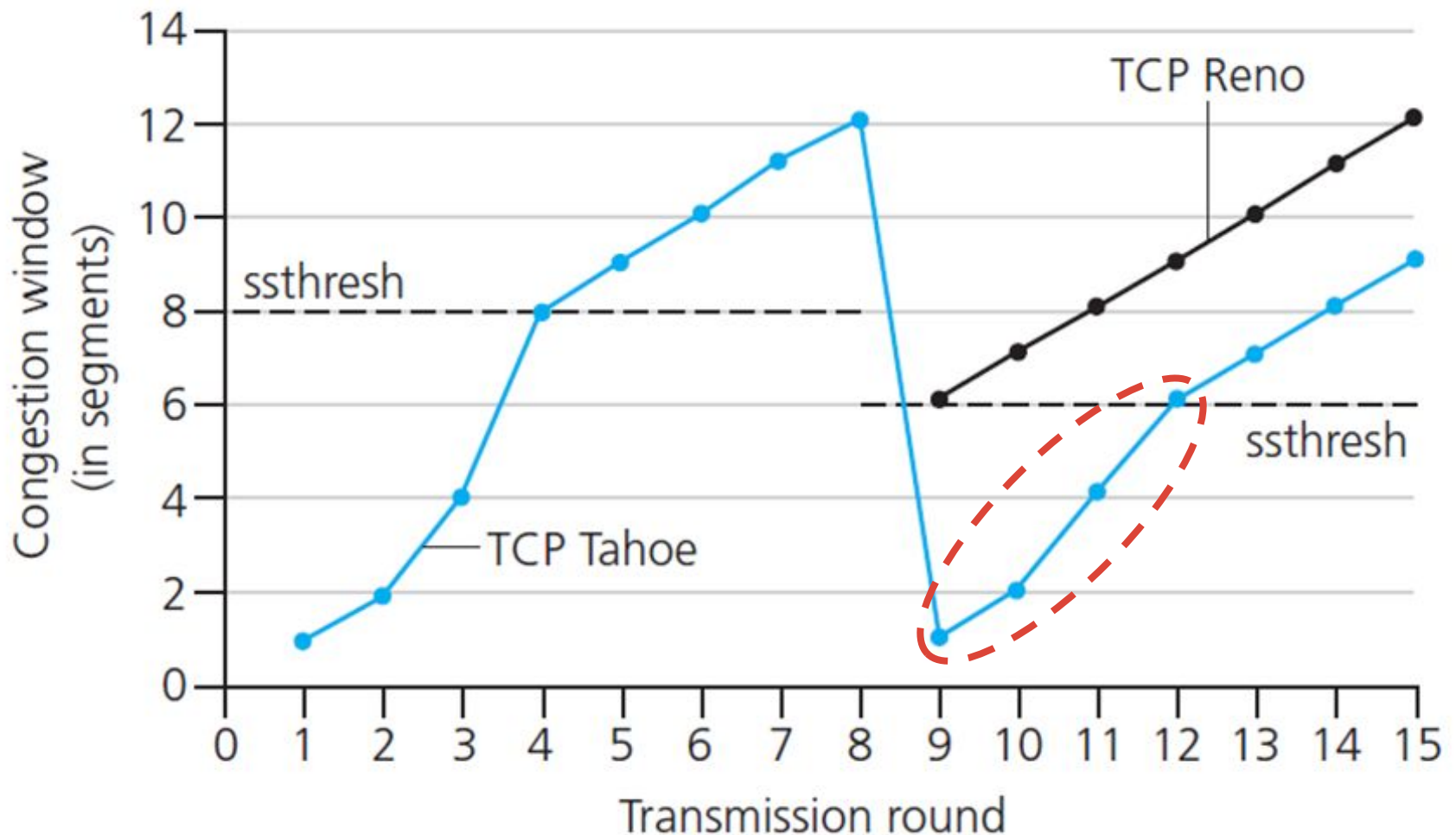
TCP : control de congestión

Algoritmo TCP Tahoe

- Le da al caso de los ACKs duplicados el mismo tratamiento que a un RTO (**Fast Retransmit** y luego **Slow Start**)

Control de Congestión: Dup. ACK

Tahoe[Dup. ACK] => Slow Start

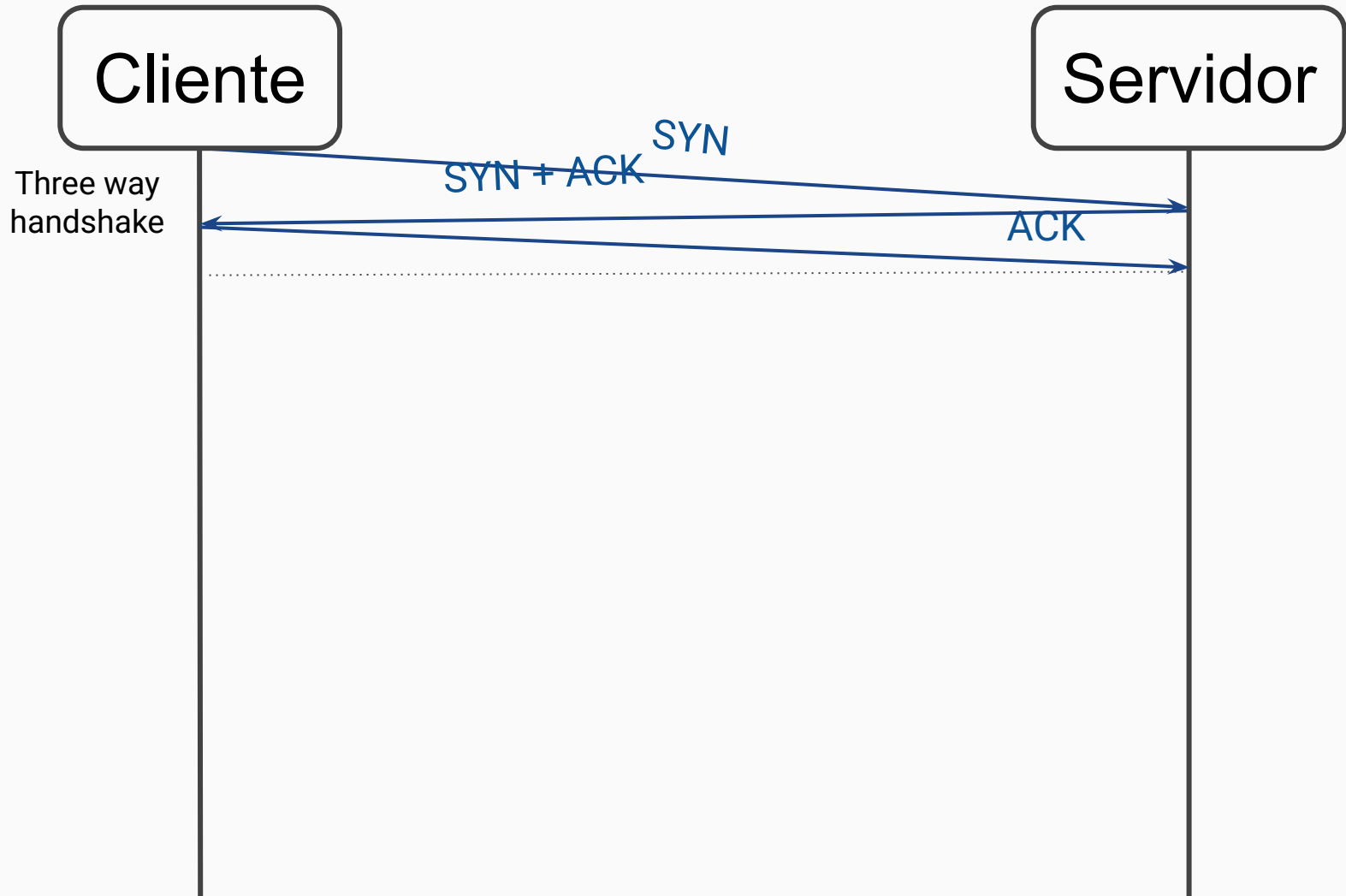


Tahoe: Un Ejemplo

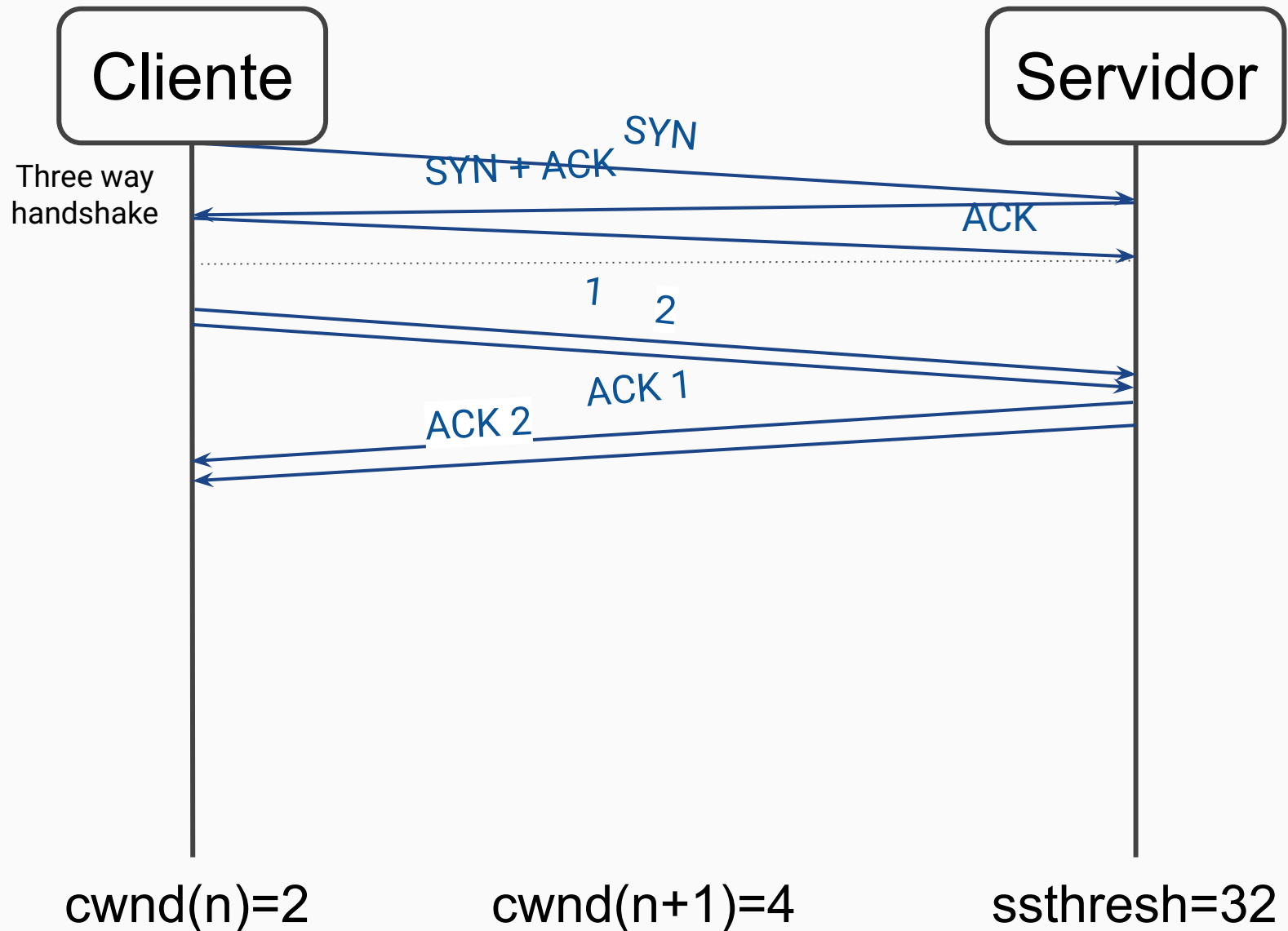
- Conexión TCP recién establecida
 - $IW = 2 \text{ MSS}$
 - $SSTHRESH = 32 \text{ MSS}$
 - Receptor siempre anuncia Window = 8 MSS
 - Se pierde el segmento de datos #8

Se envía un archivo de 18 MSS

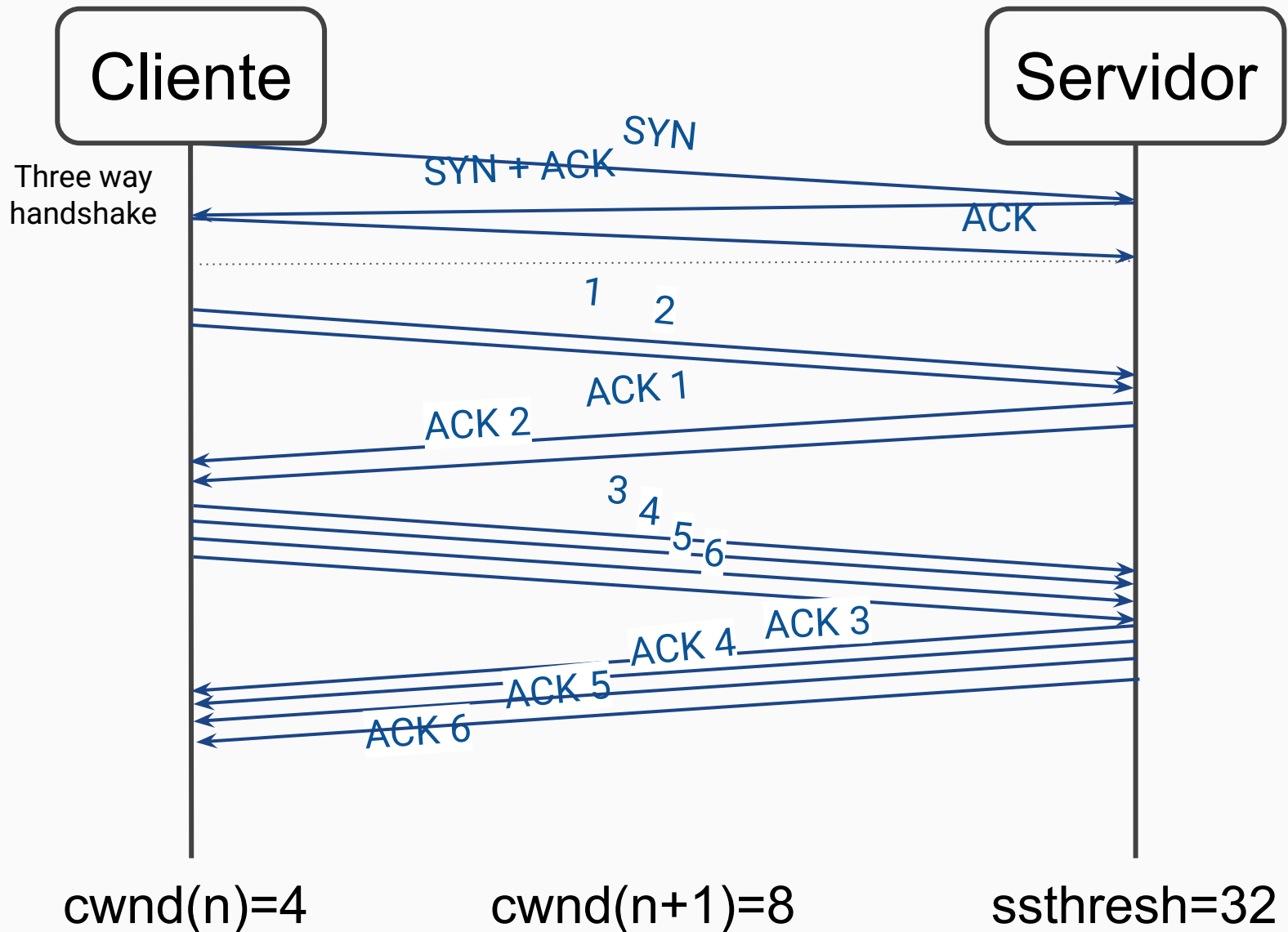
Three way handshake



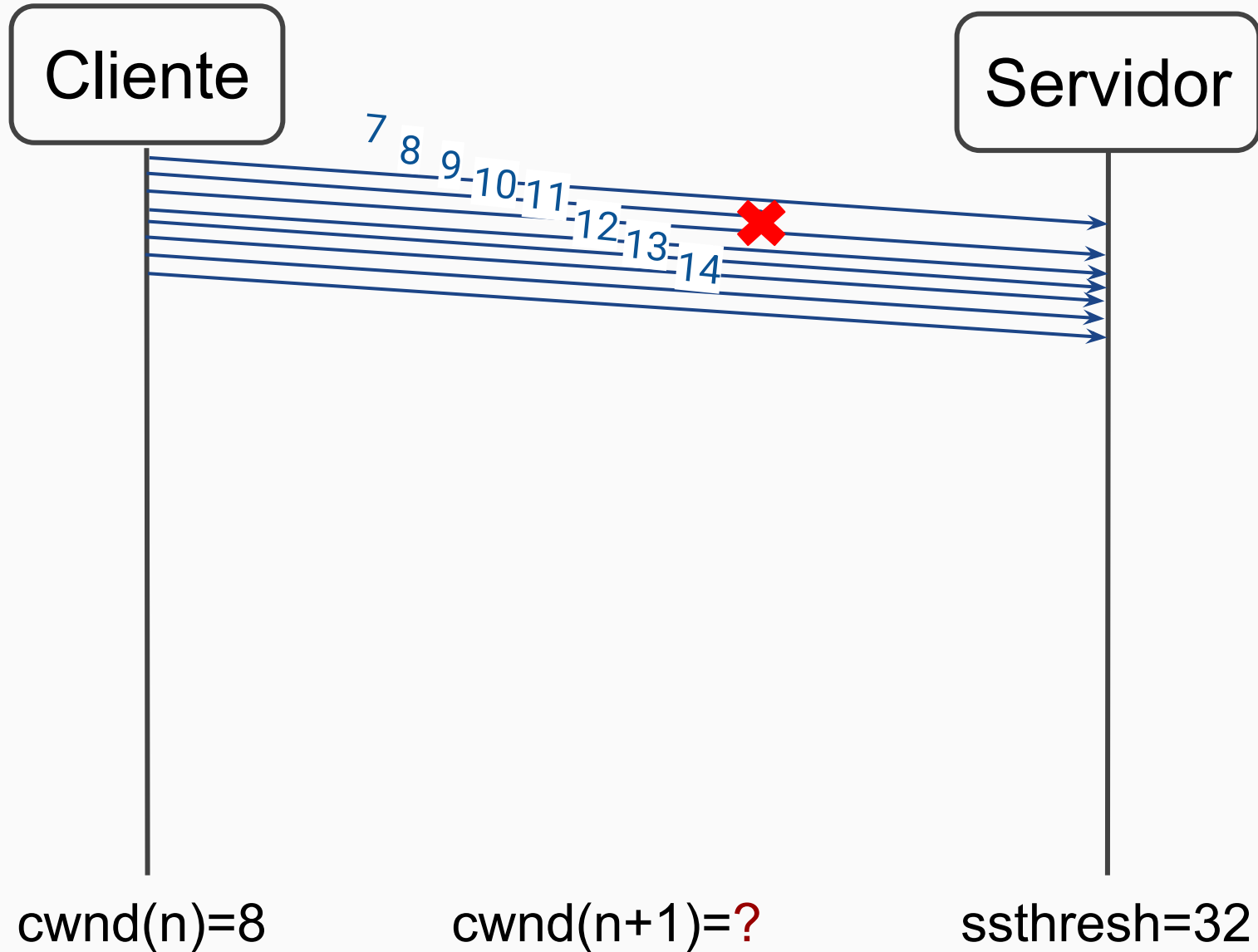
Slow start



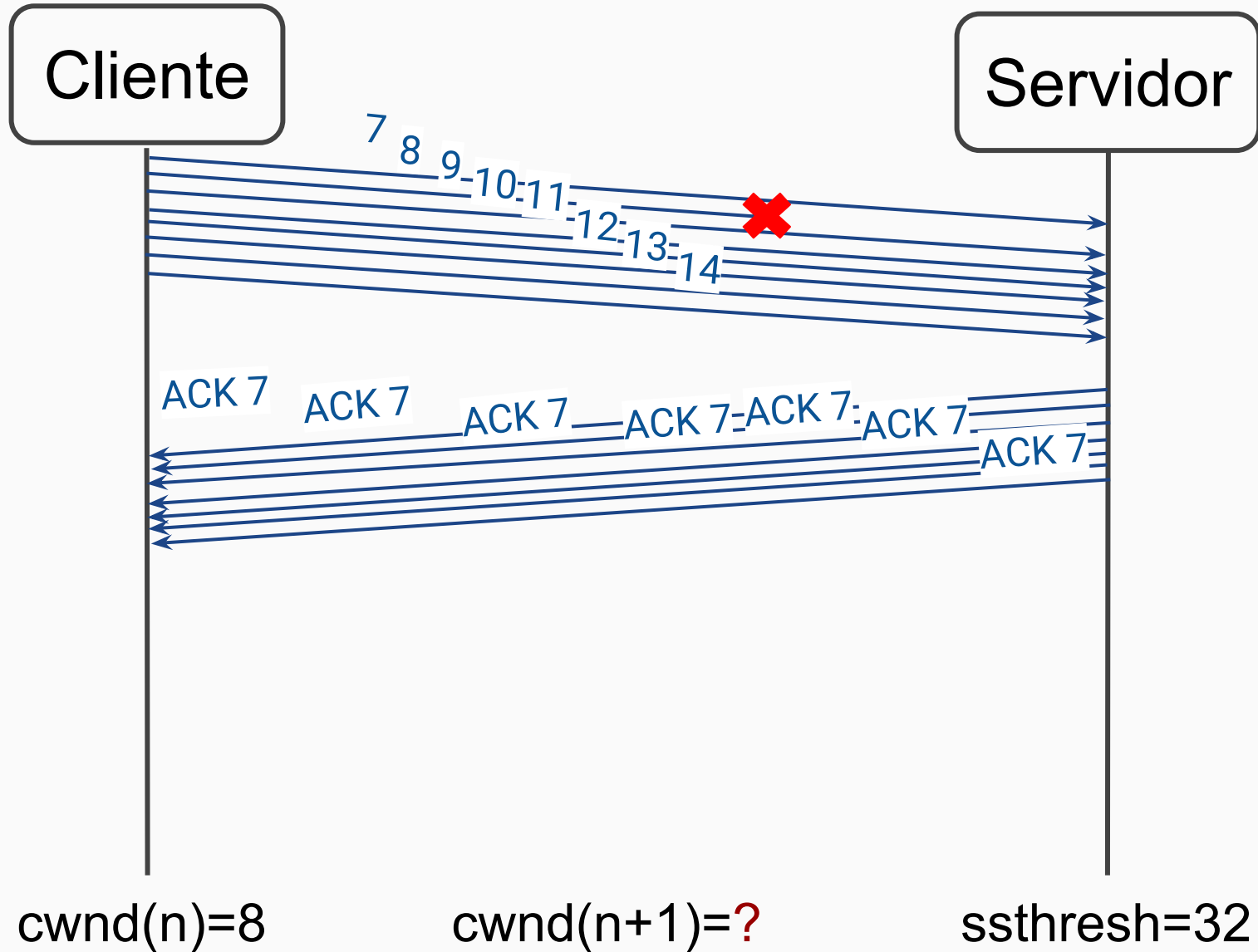
Slow start



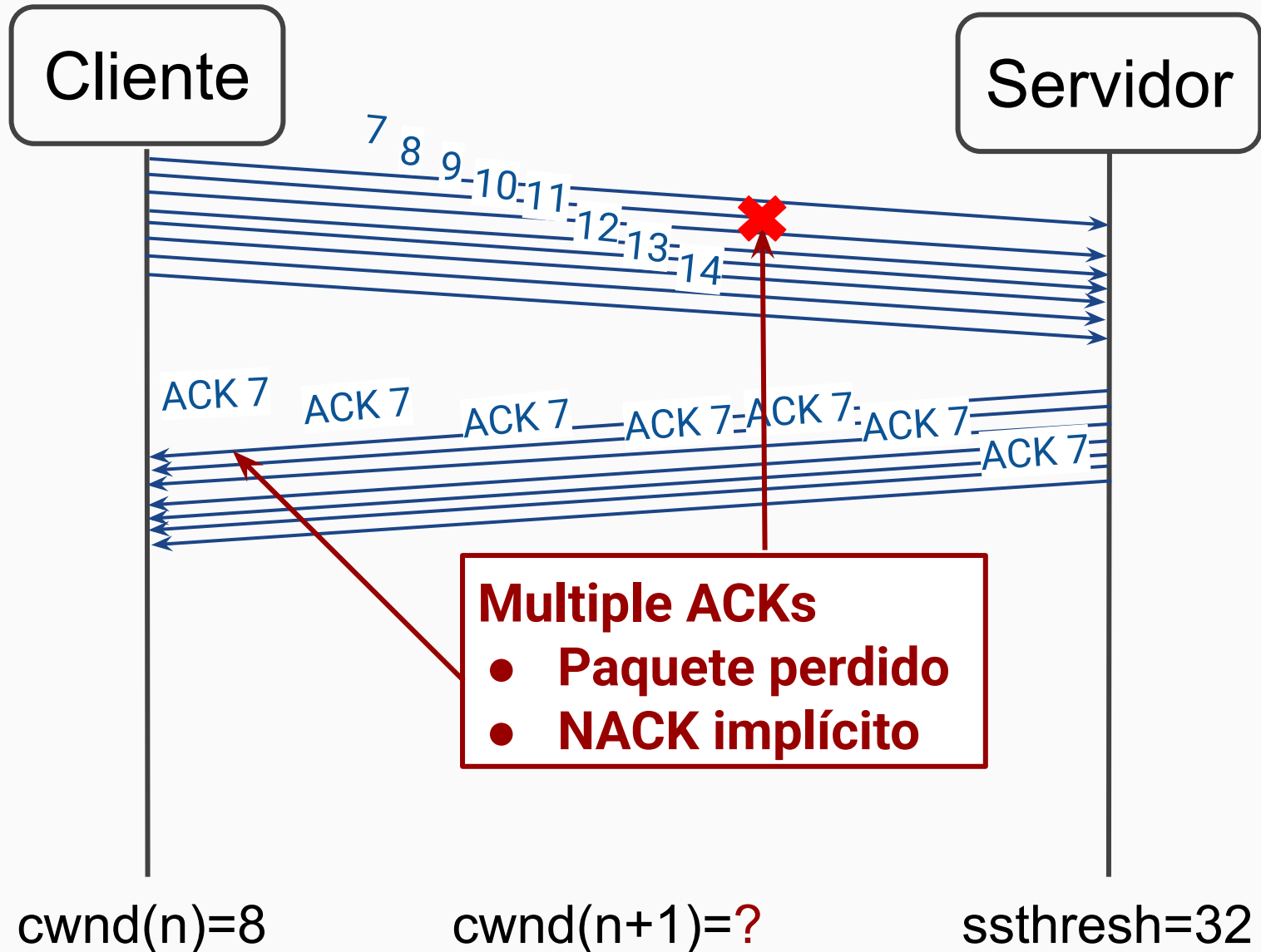
Slow start



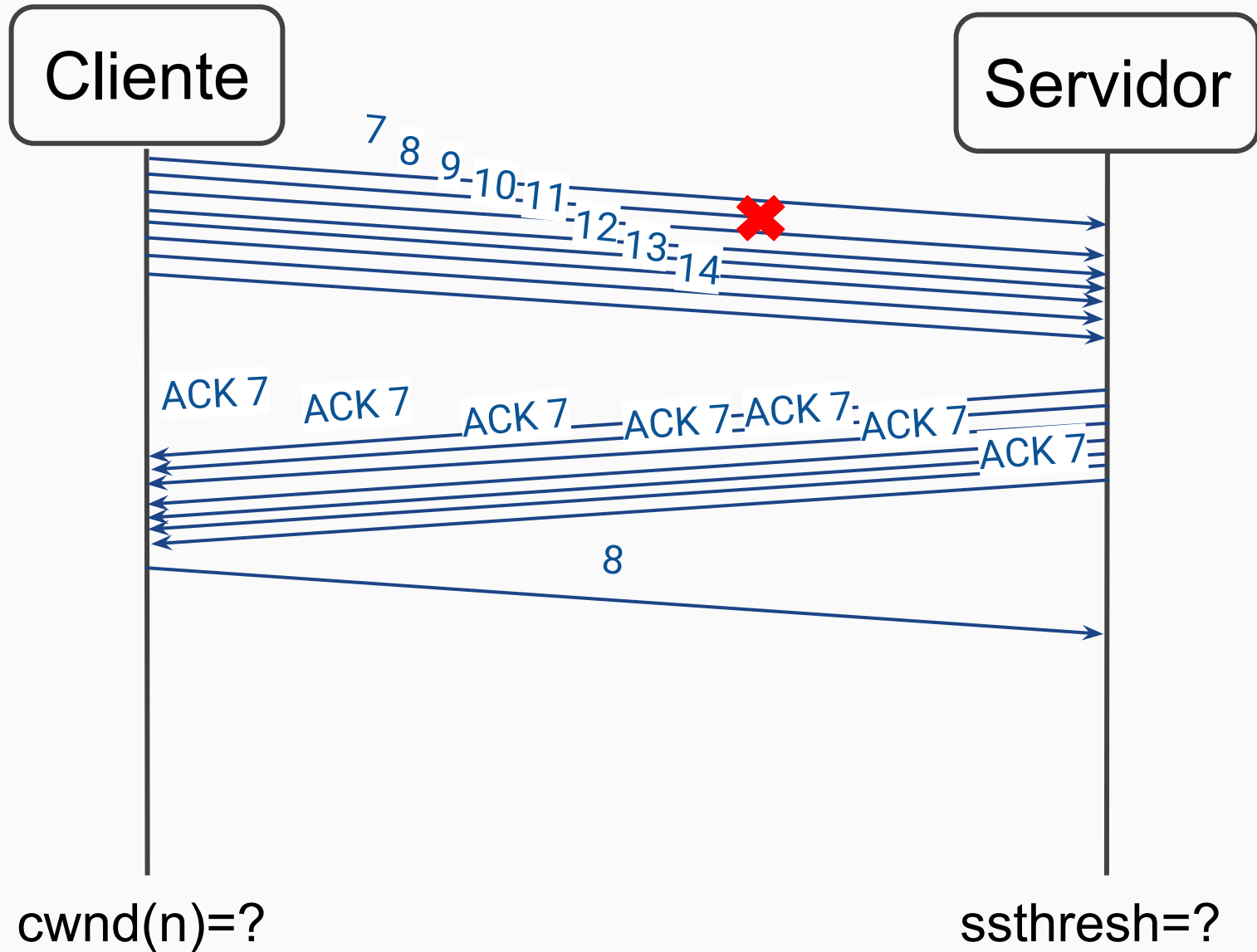
Slow start



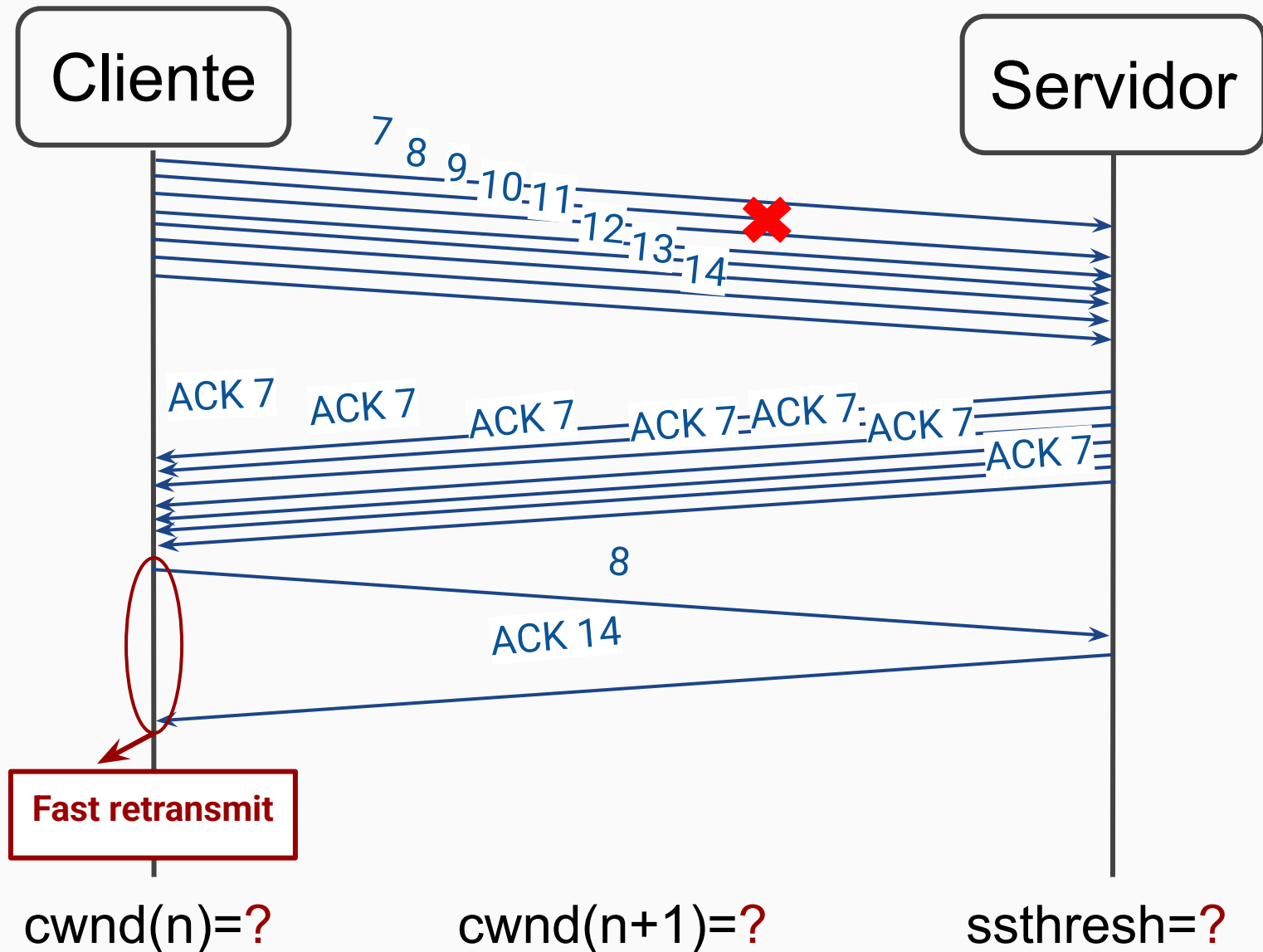
Slow start



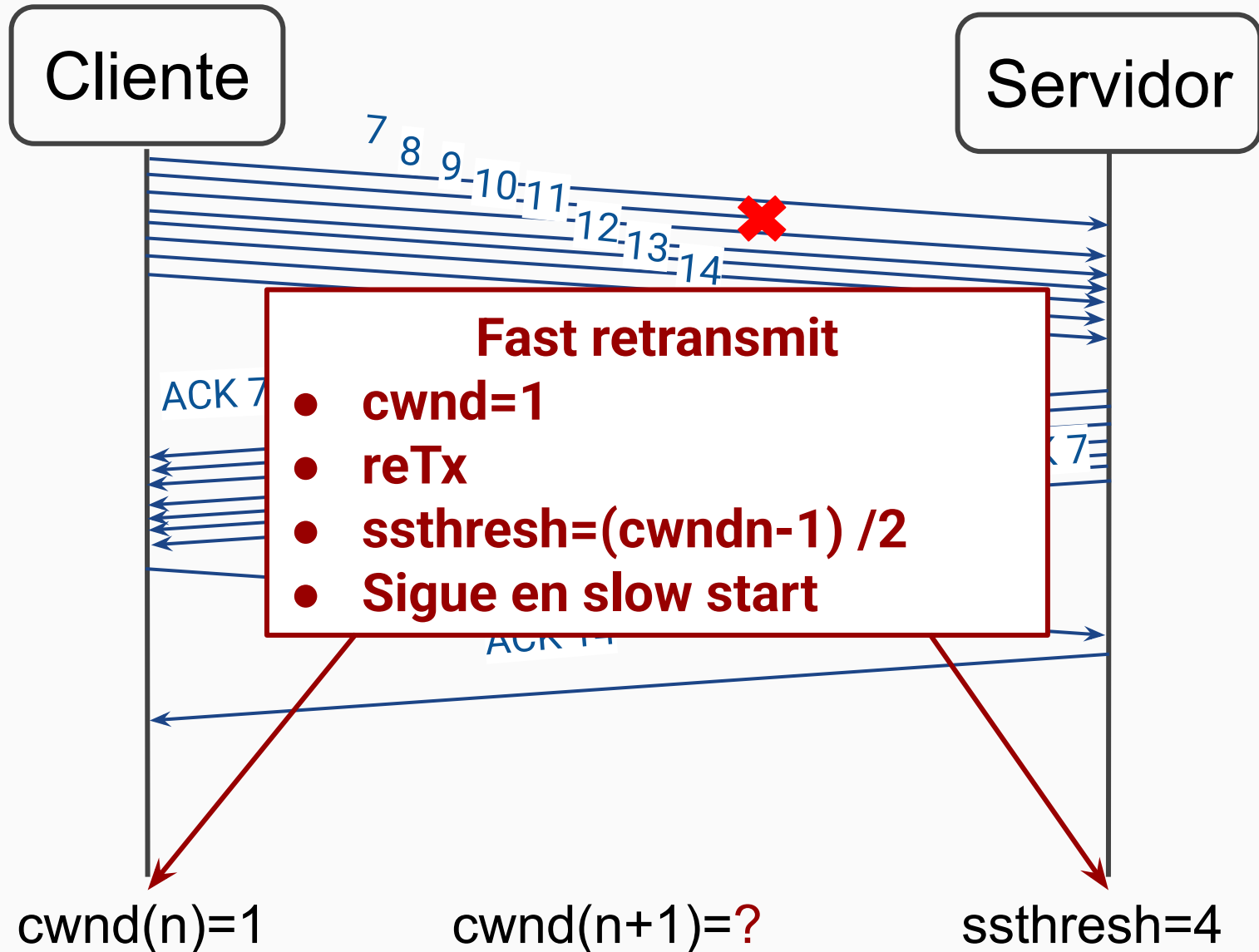
Slow start



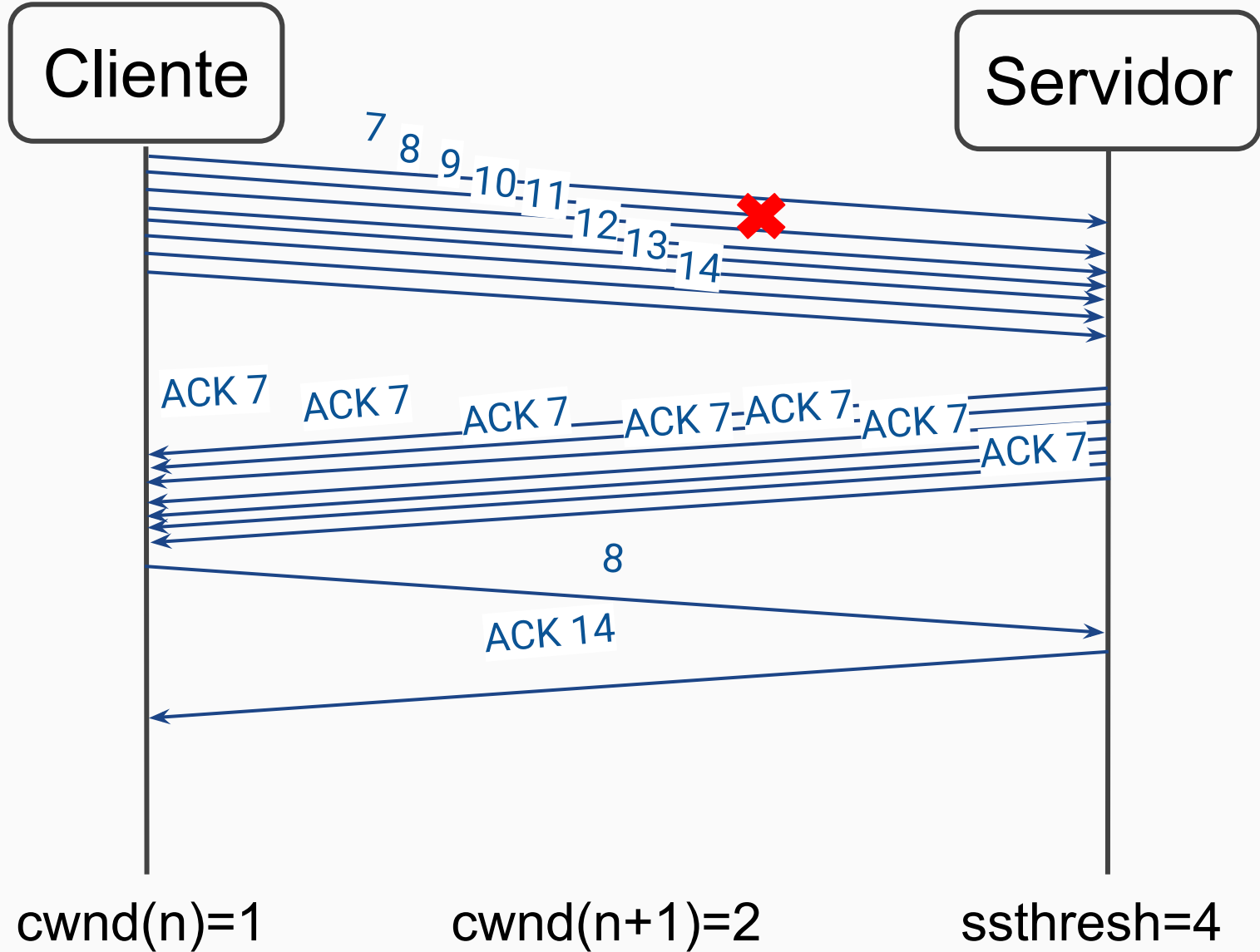
Fast Retransmit



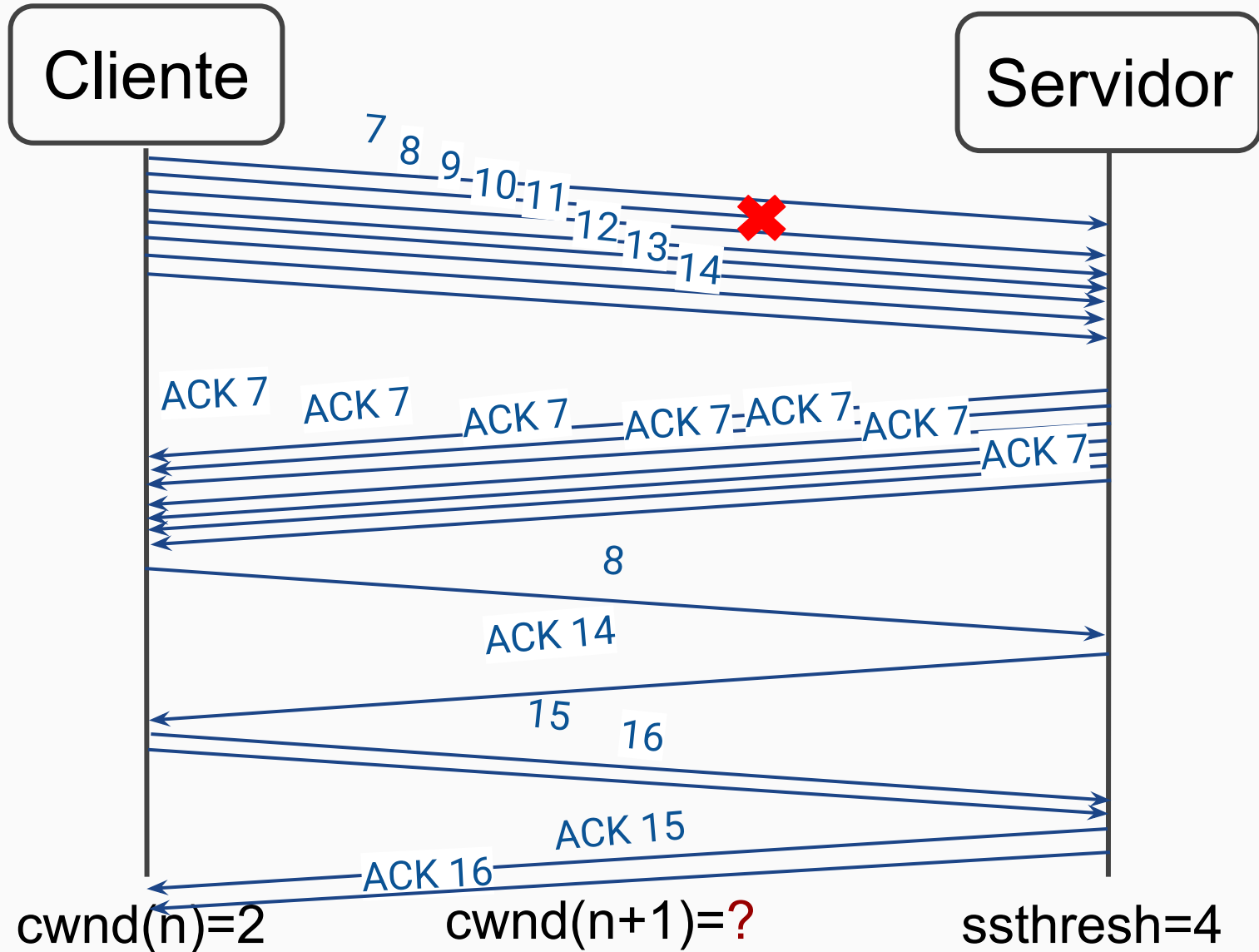
Fast Retransmit



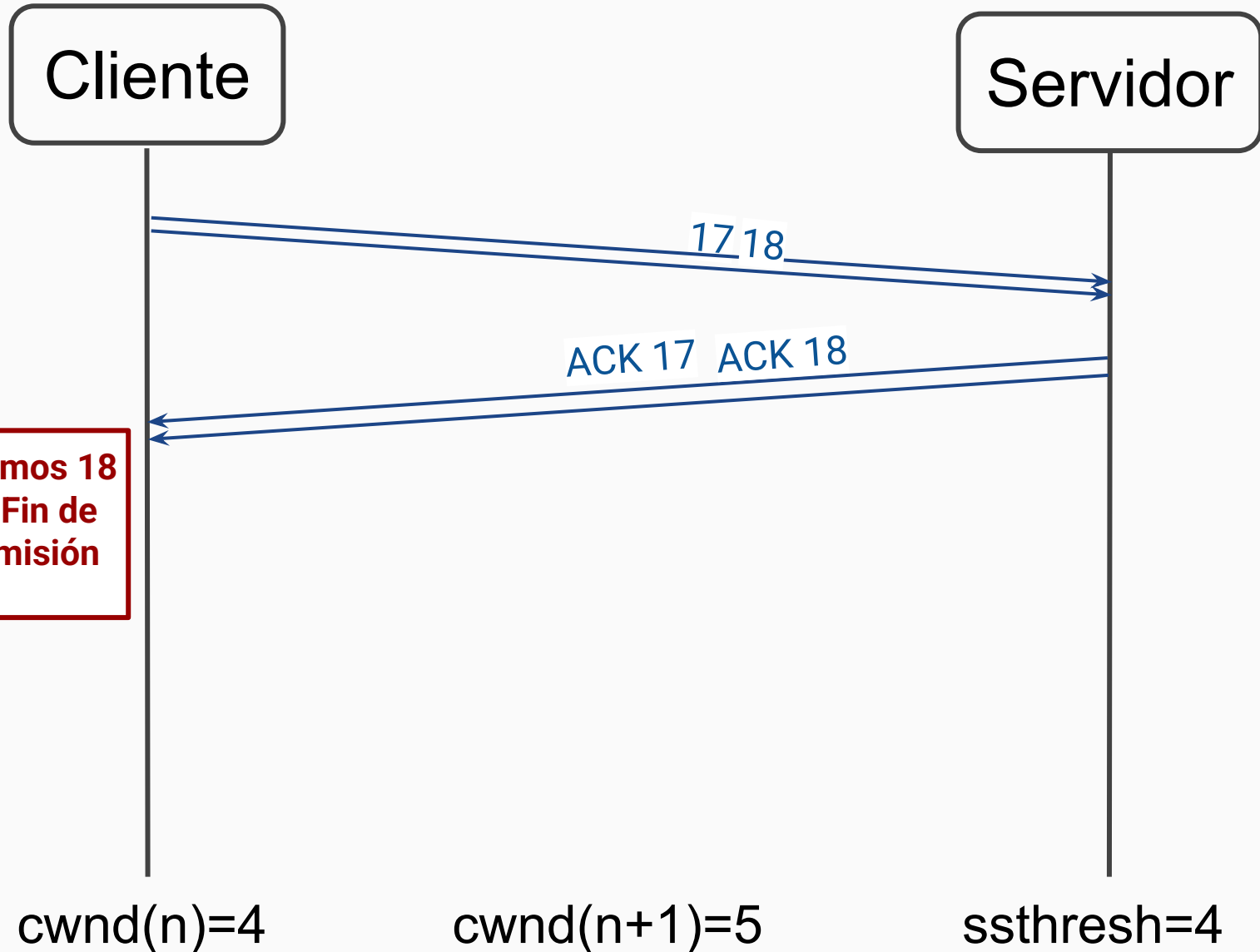
Slow start



Slow start



Slow start



ACKs Duplicados

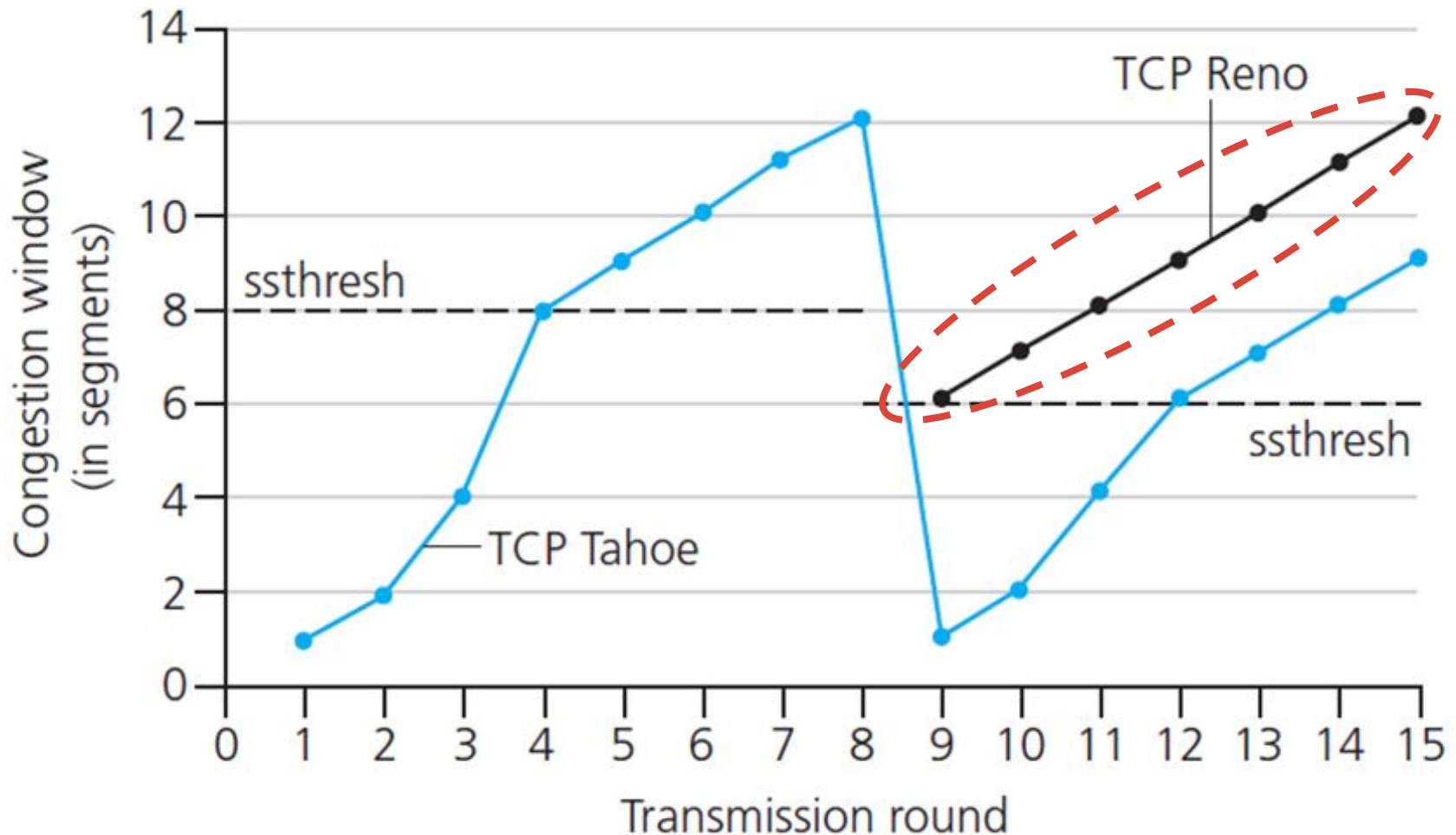
Reno

Pérdida de paquetes

- **Ahora, veamos el caso de Reno**
- Supongamos que se pierde **UN SOLO** segmento de la ráfaga en cuestión.
- Y además, que estamos usando el algoritmo de control de congestión de **Reno**

Control de Congestión: Dup. ACK

Reno[Dup. ACK] => Baja a la mitad (en CA)



TCP : control de congestión

Algoritmo TCP Reno

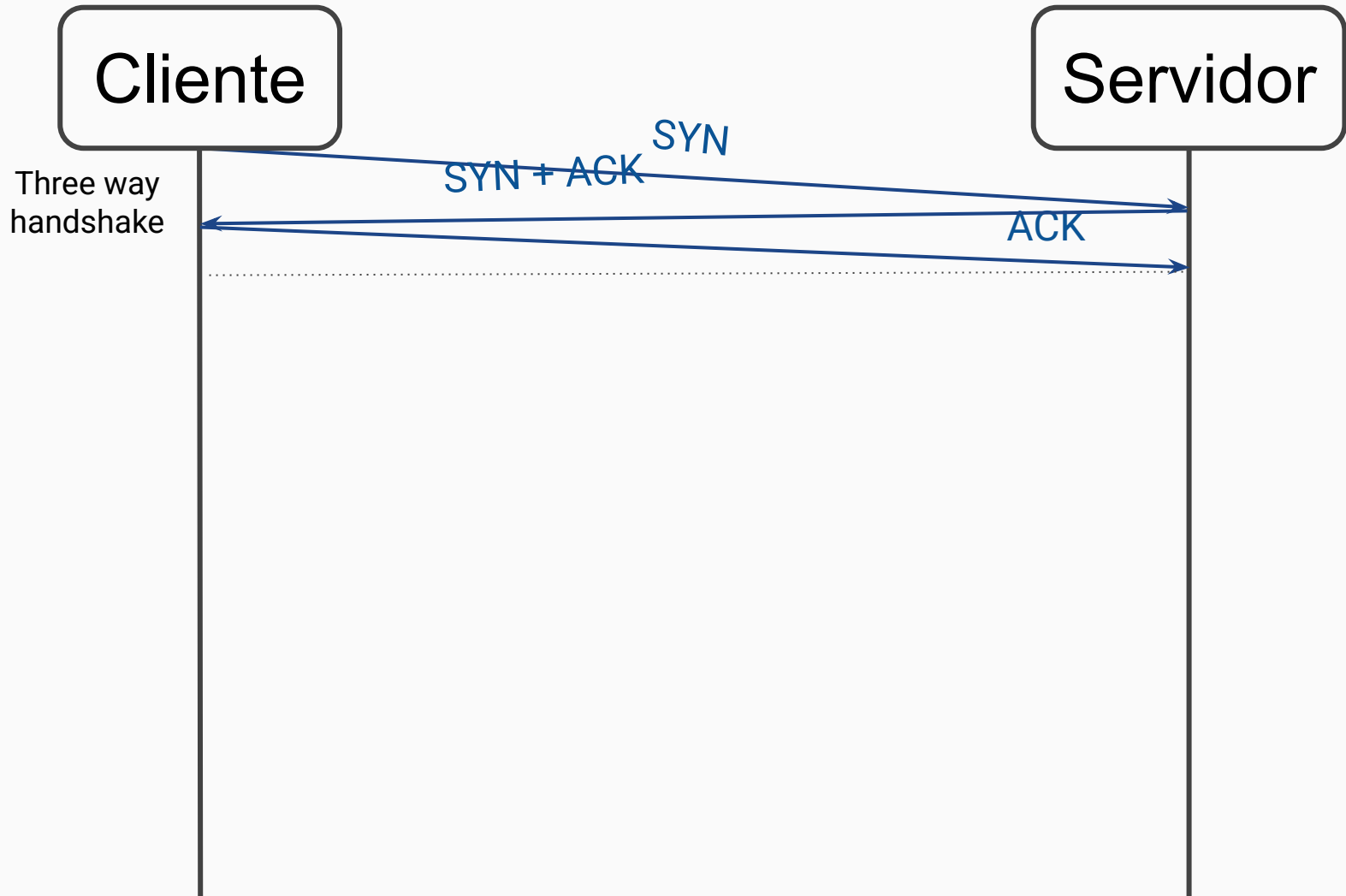
- Cuando se reciben 4 ACKs iguales, se retransmite el siguiente segmento
- $\text{cwnd}(n+1) = \text{cwnd}(n) / 2$
- $\text{ssthresh} = \text{cwnd}(n) / 2$
- Se continúa con la fase de **CA** en vez de pasar a **Slow Start**
- Esto se denomina **Fast Recovery**

Reno: Mismo Ejemplo

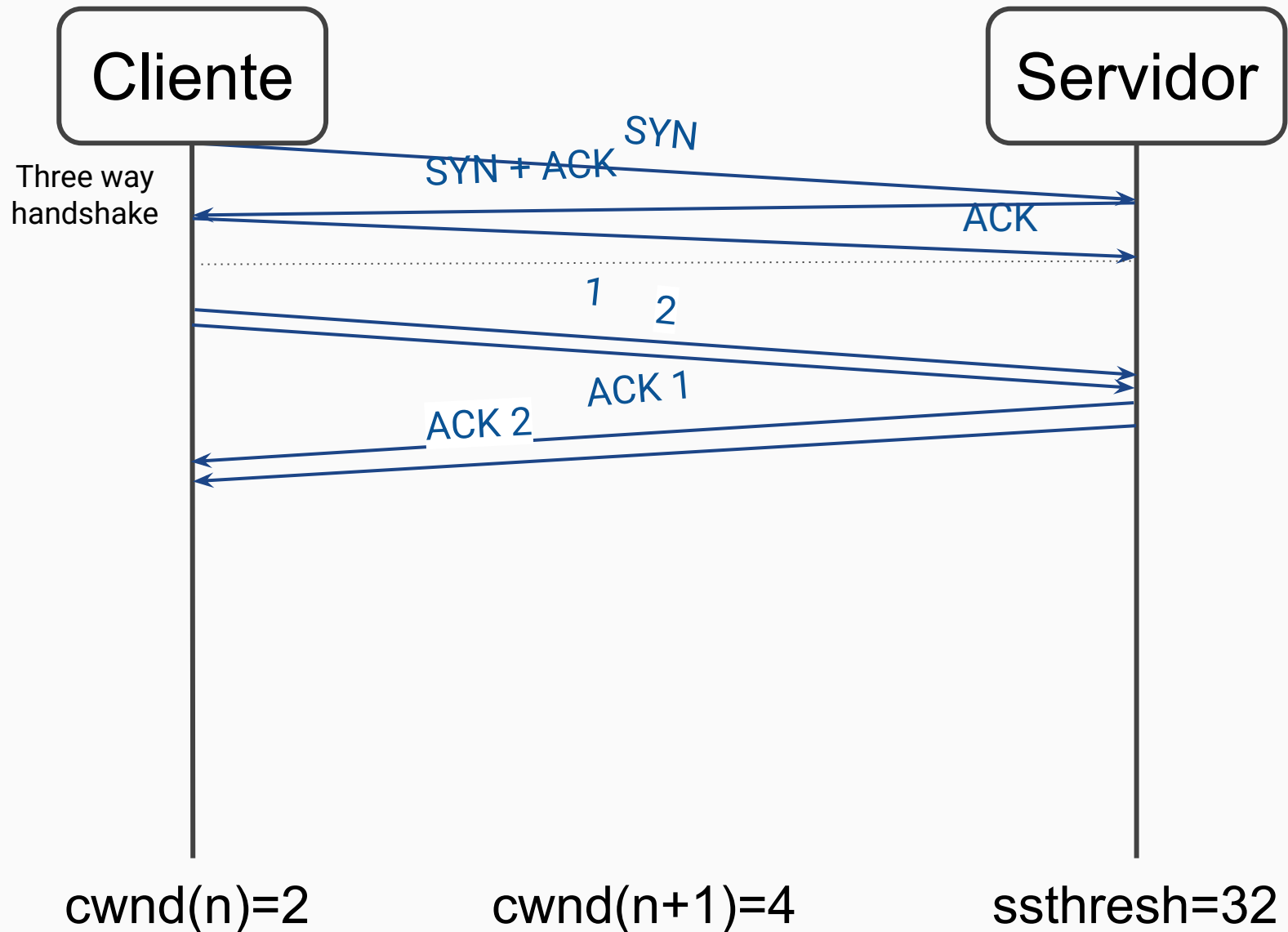
- Conexión TCP recién establecida
 - $IW = 2 \text{ MSS}$
 - $SSTHRESH = 32 \text{ MSS}$
 - Receptor siempre anuncia Window = 8 MSS
 - Se pierde el segmento de datos #8

Se envía un archivo de 18 MSS

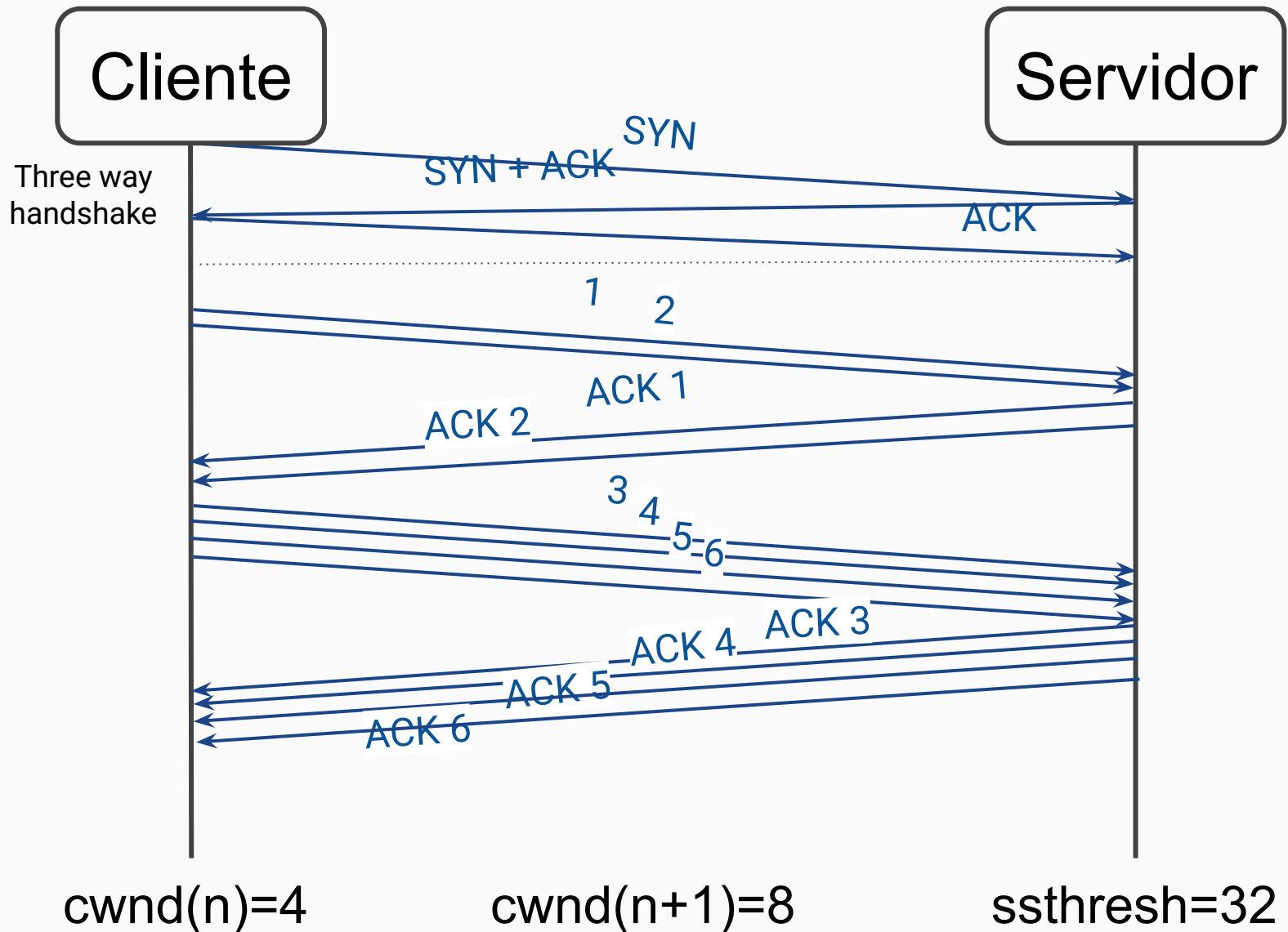
Three way handshake



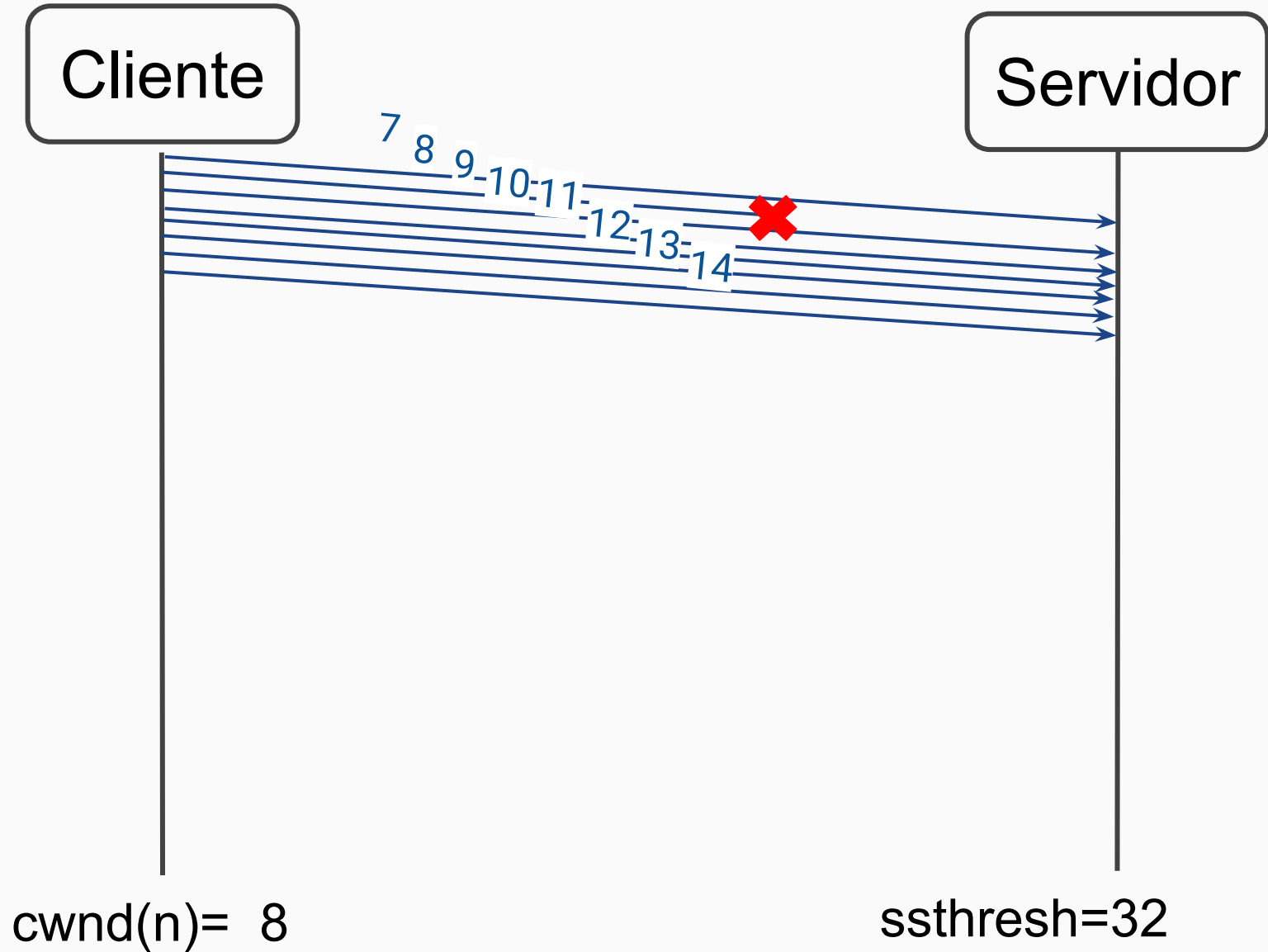
Slow start



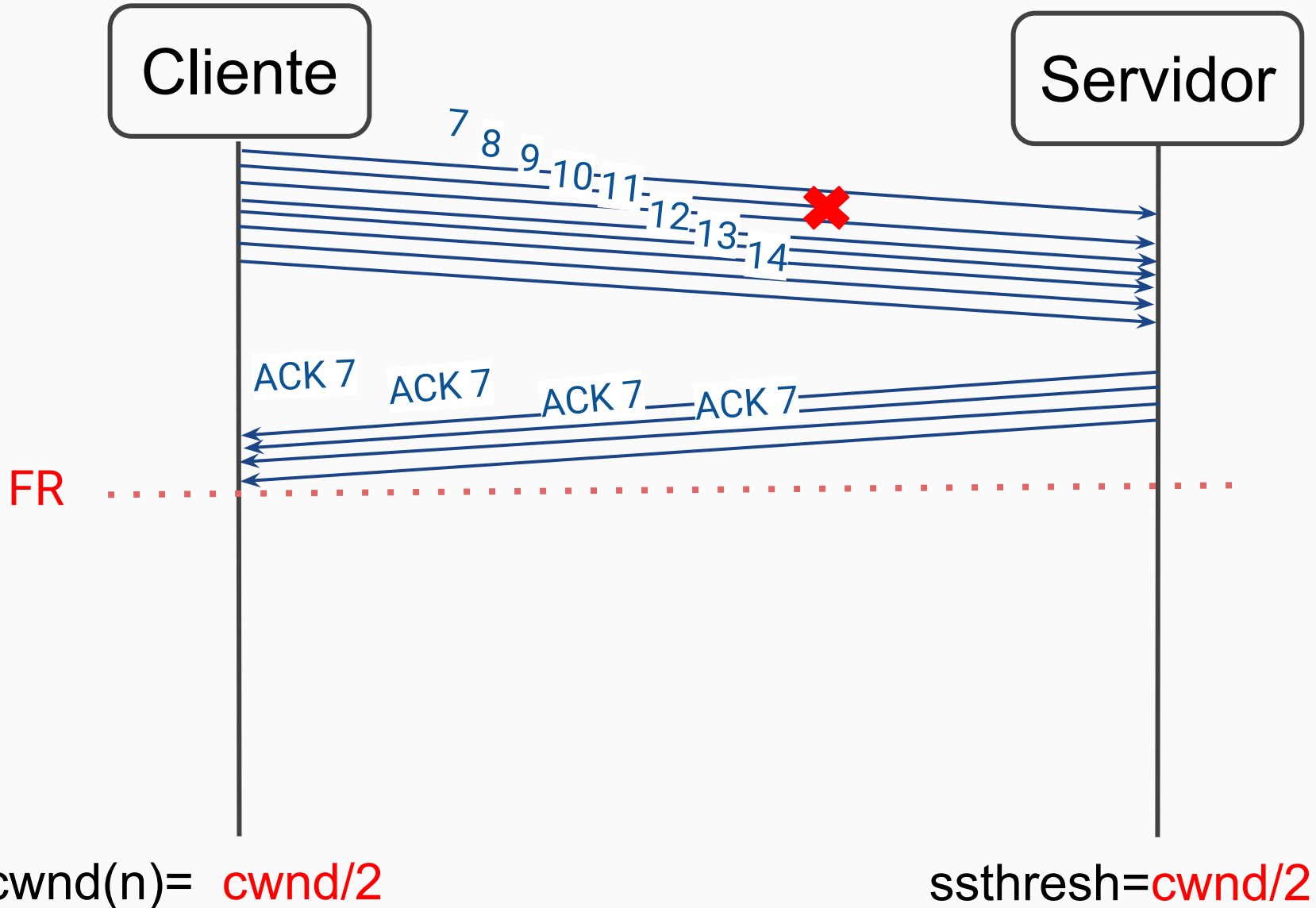
Slow start



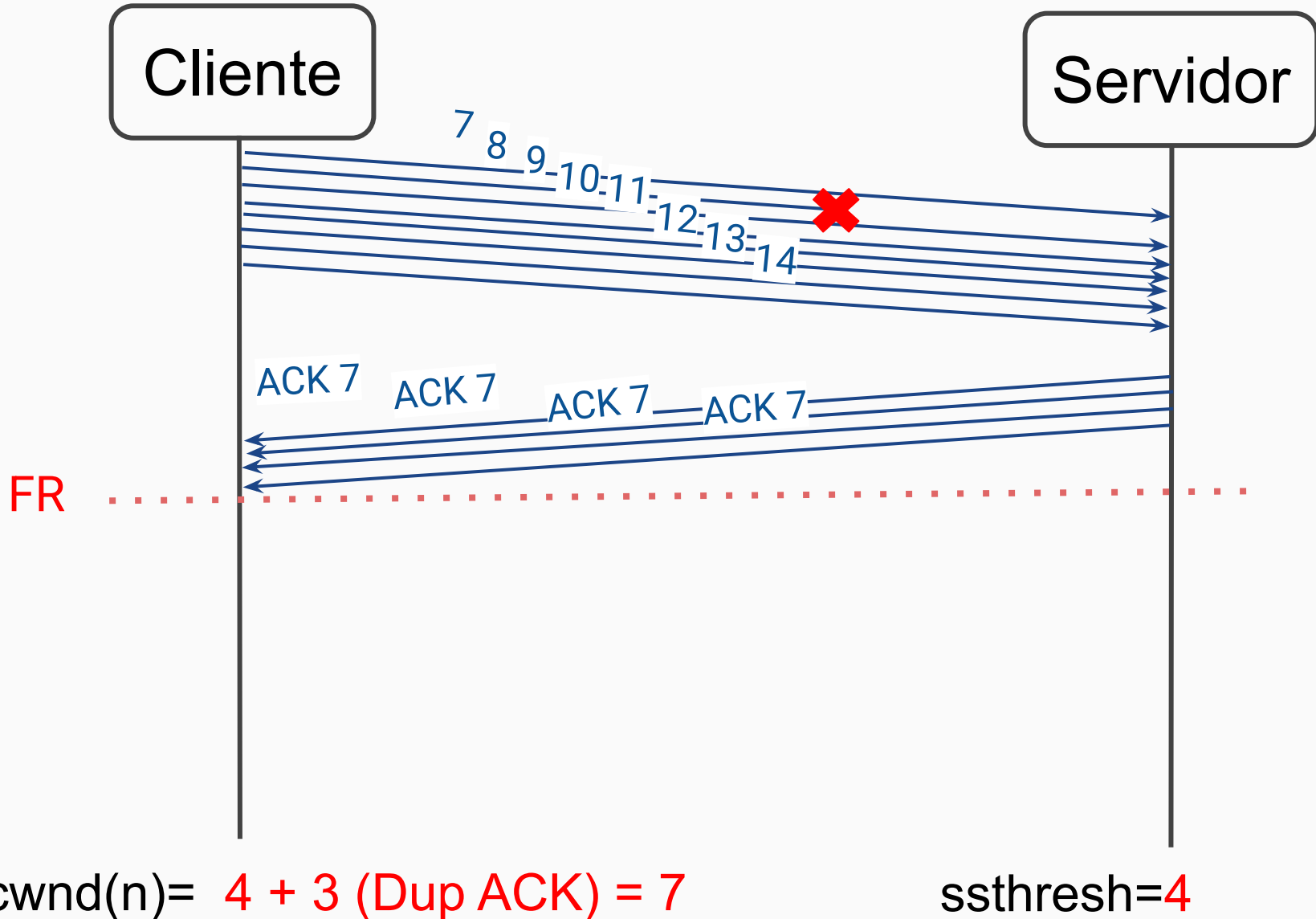
Slow start



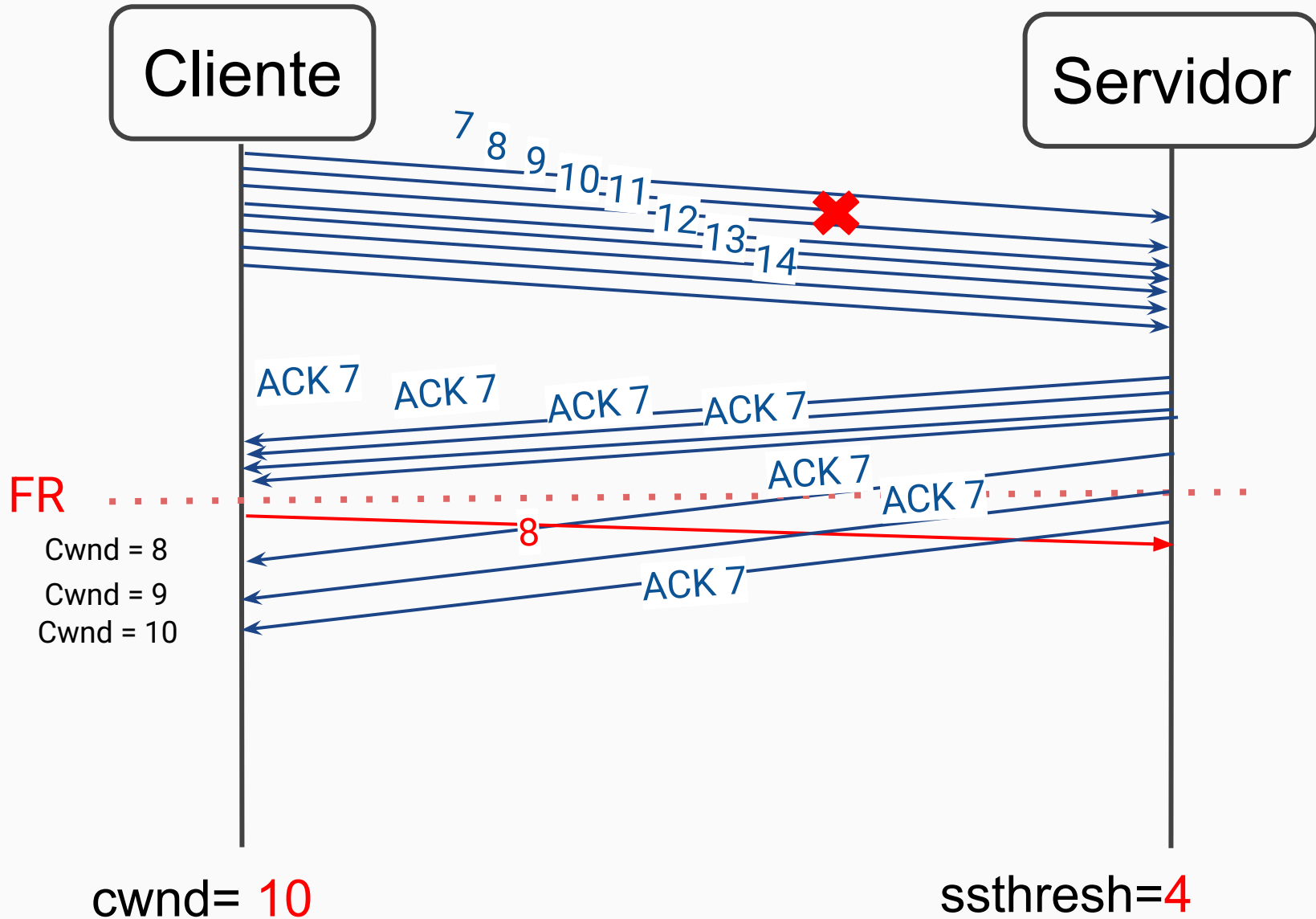
Slow start



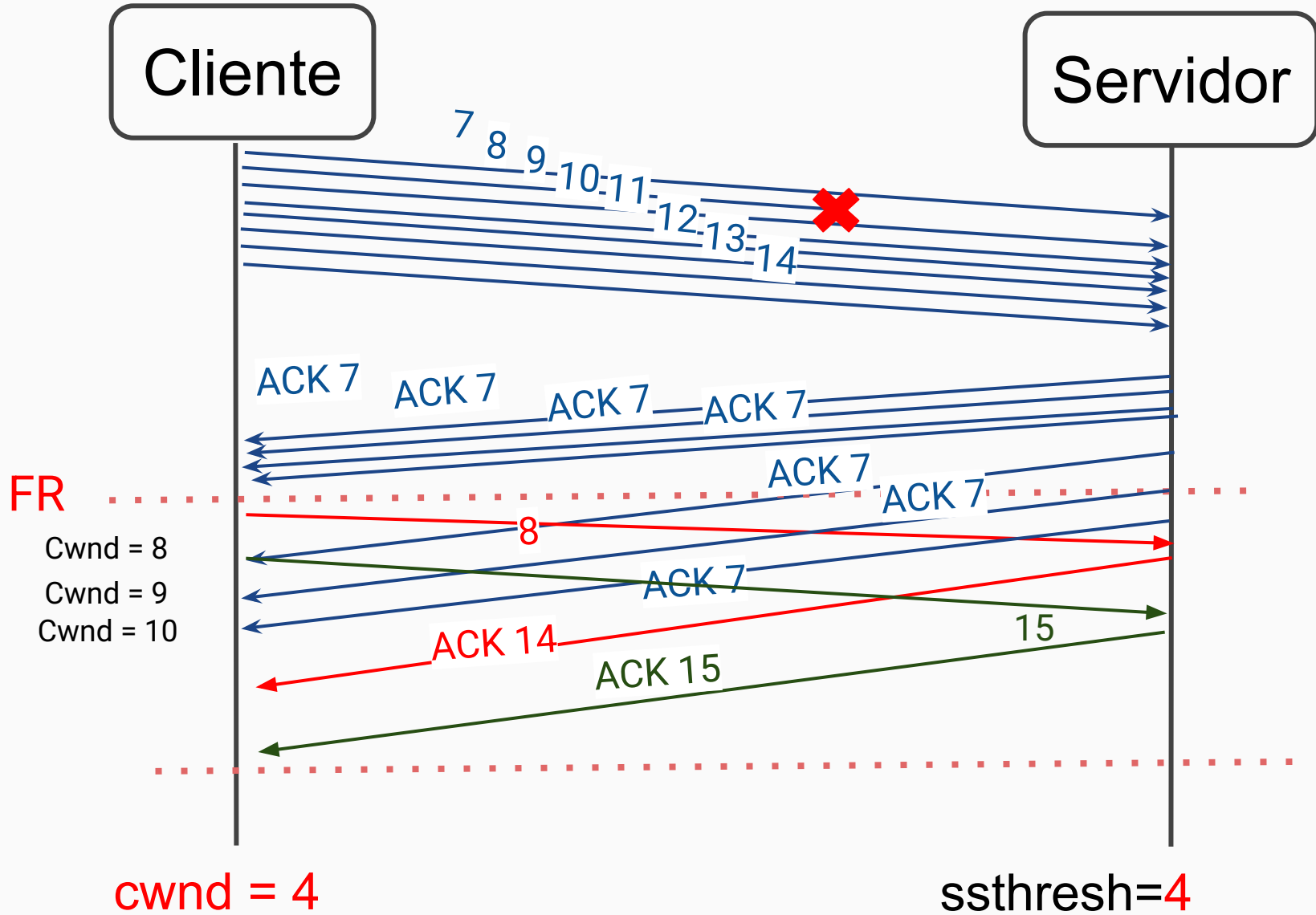
Slow start



Slow start



Slow start

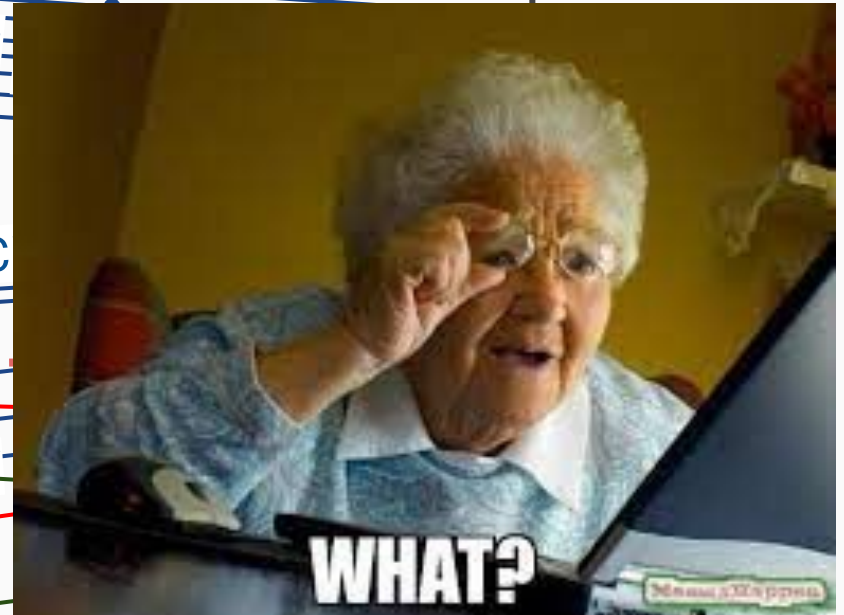
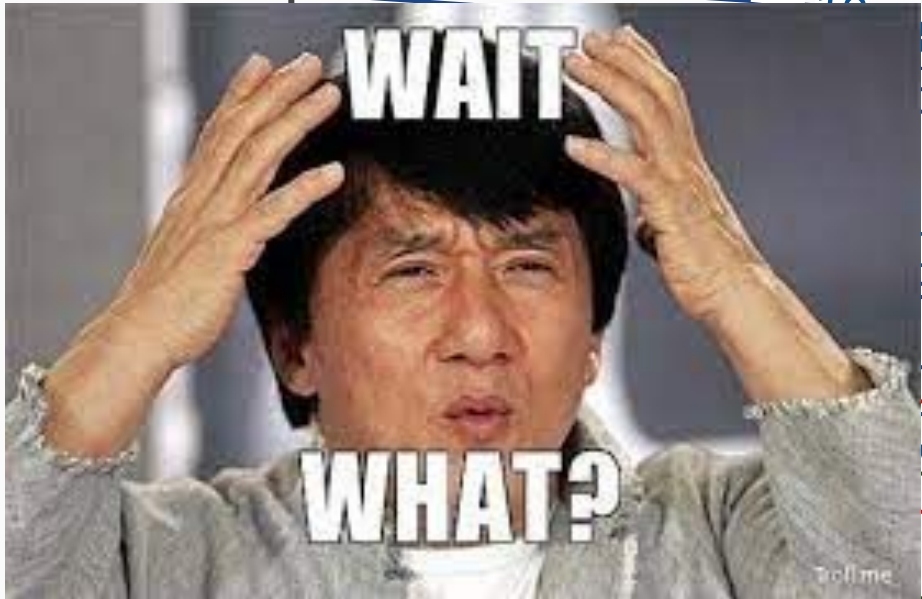


Slow start

Cliente

Servidor

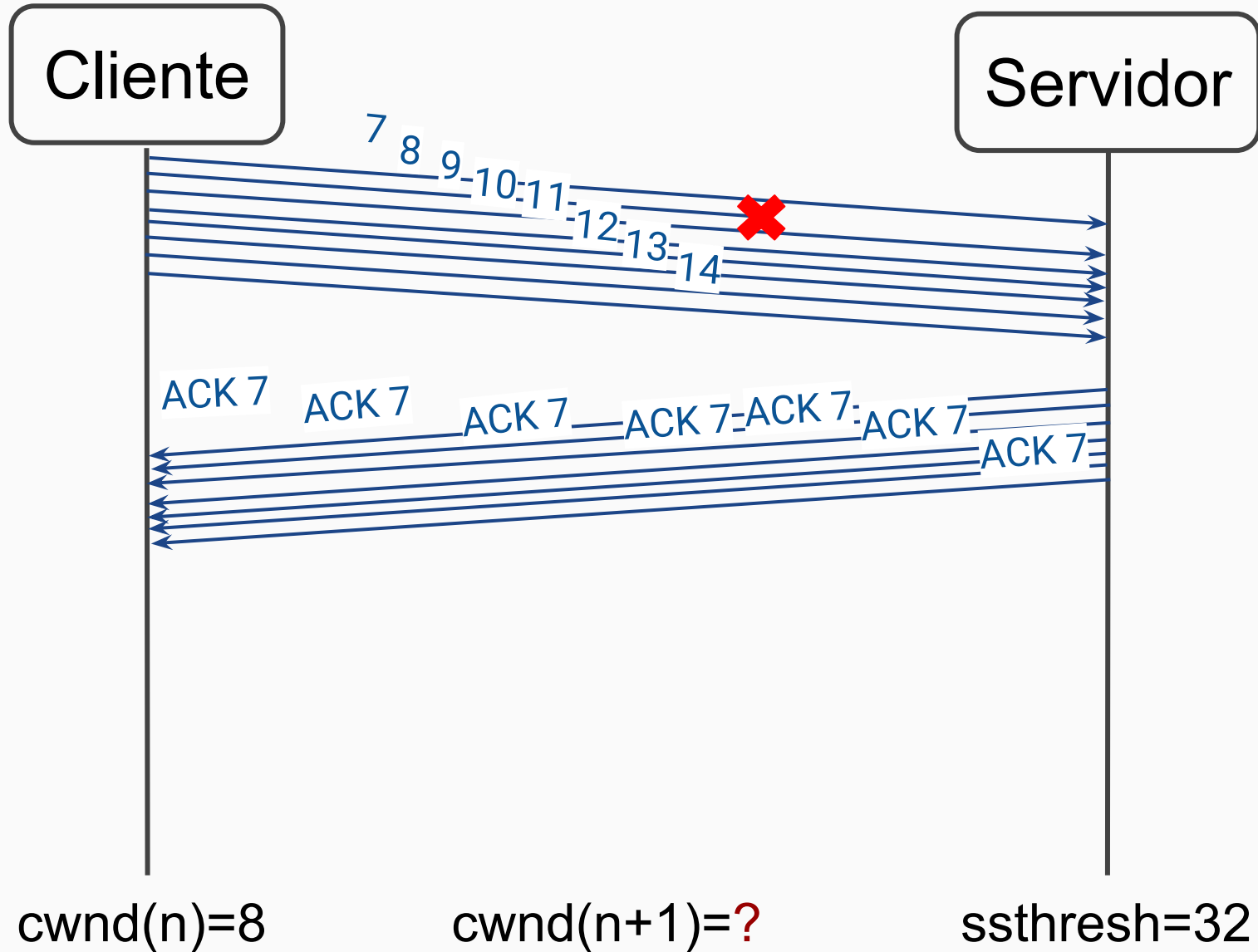
7 8 9 10 11 12



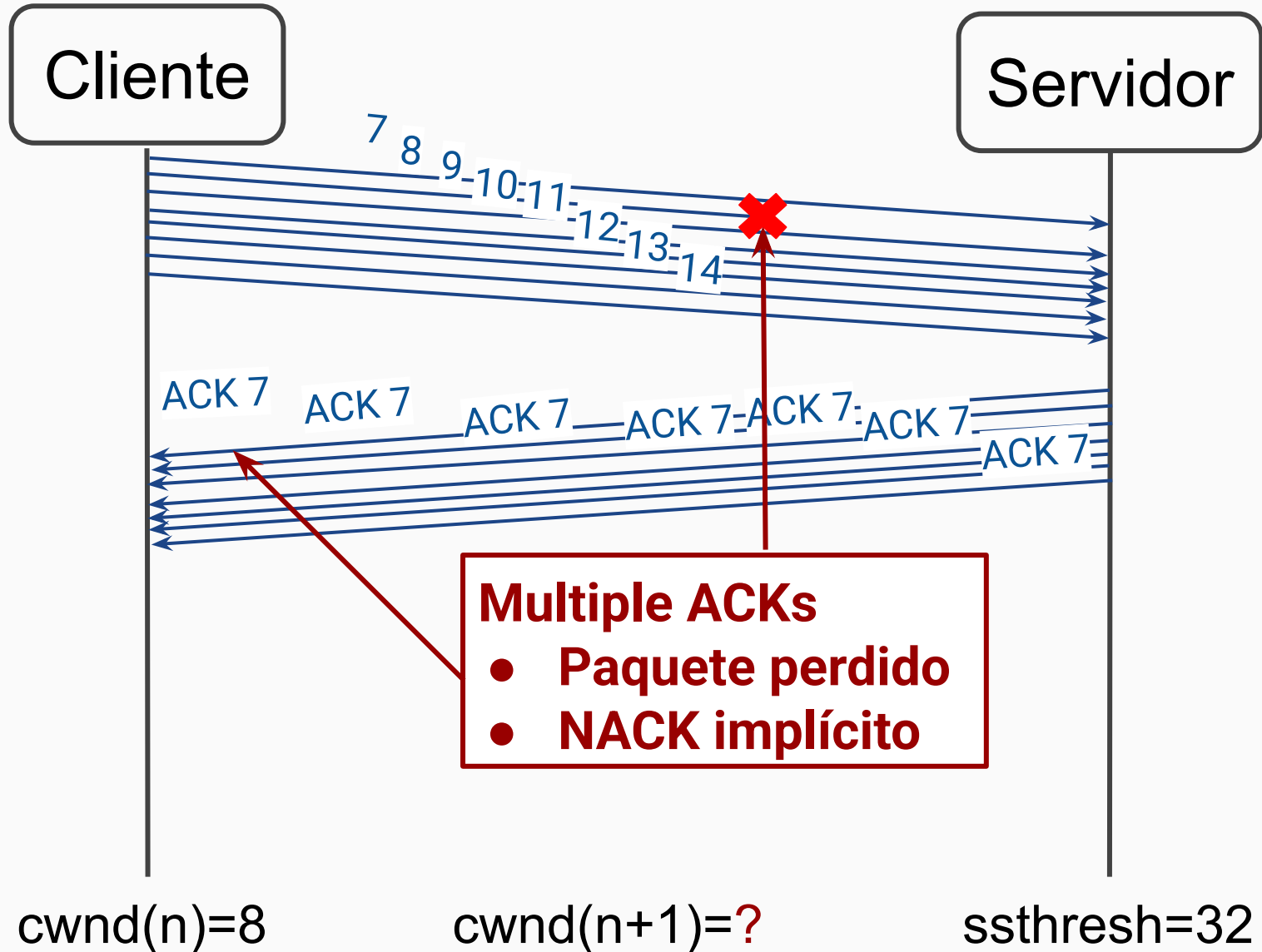
$cwnd = 4$

$ssthresh = 4$

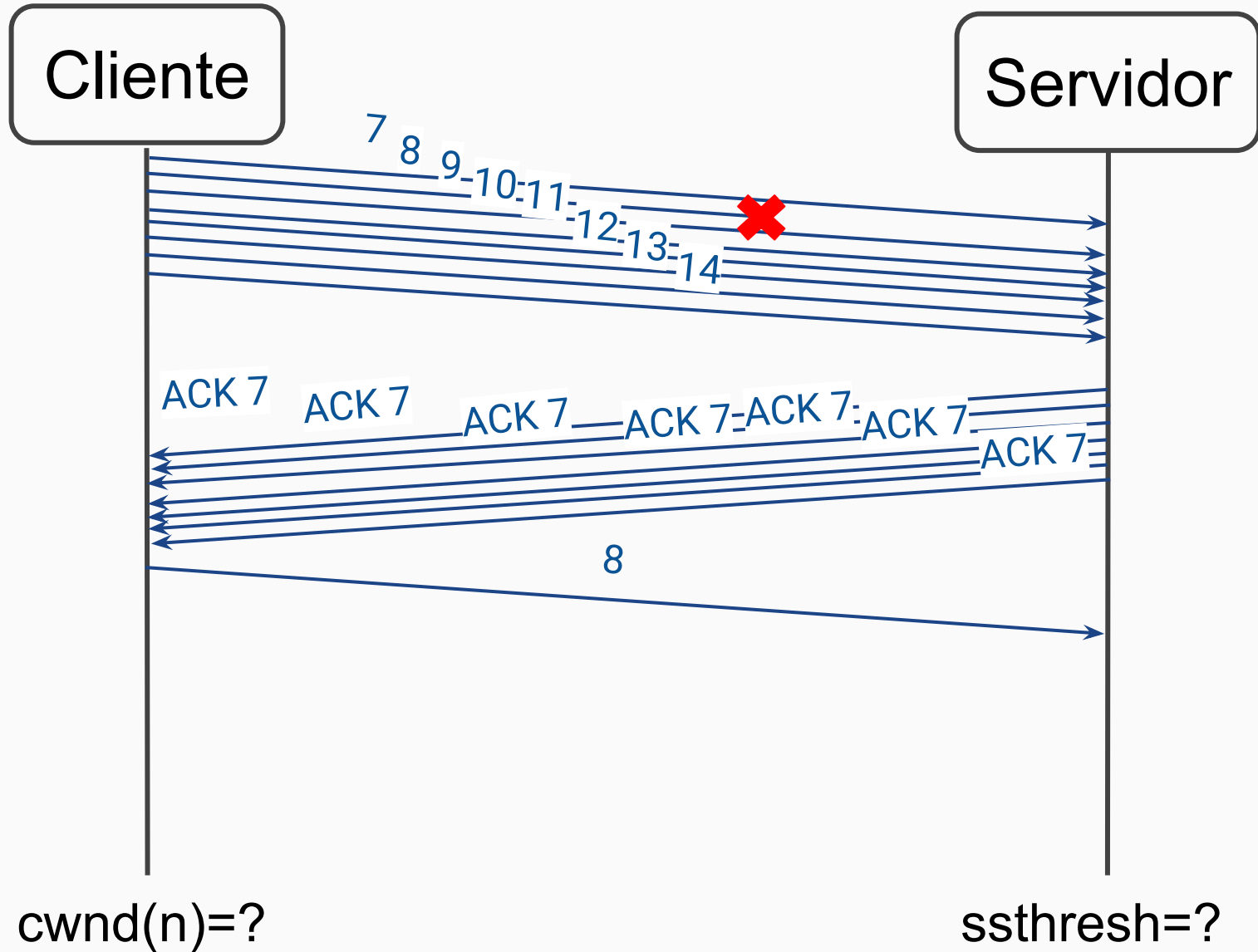
Slow start



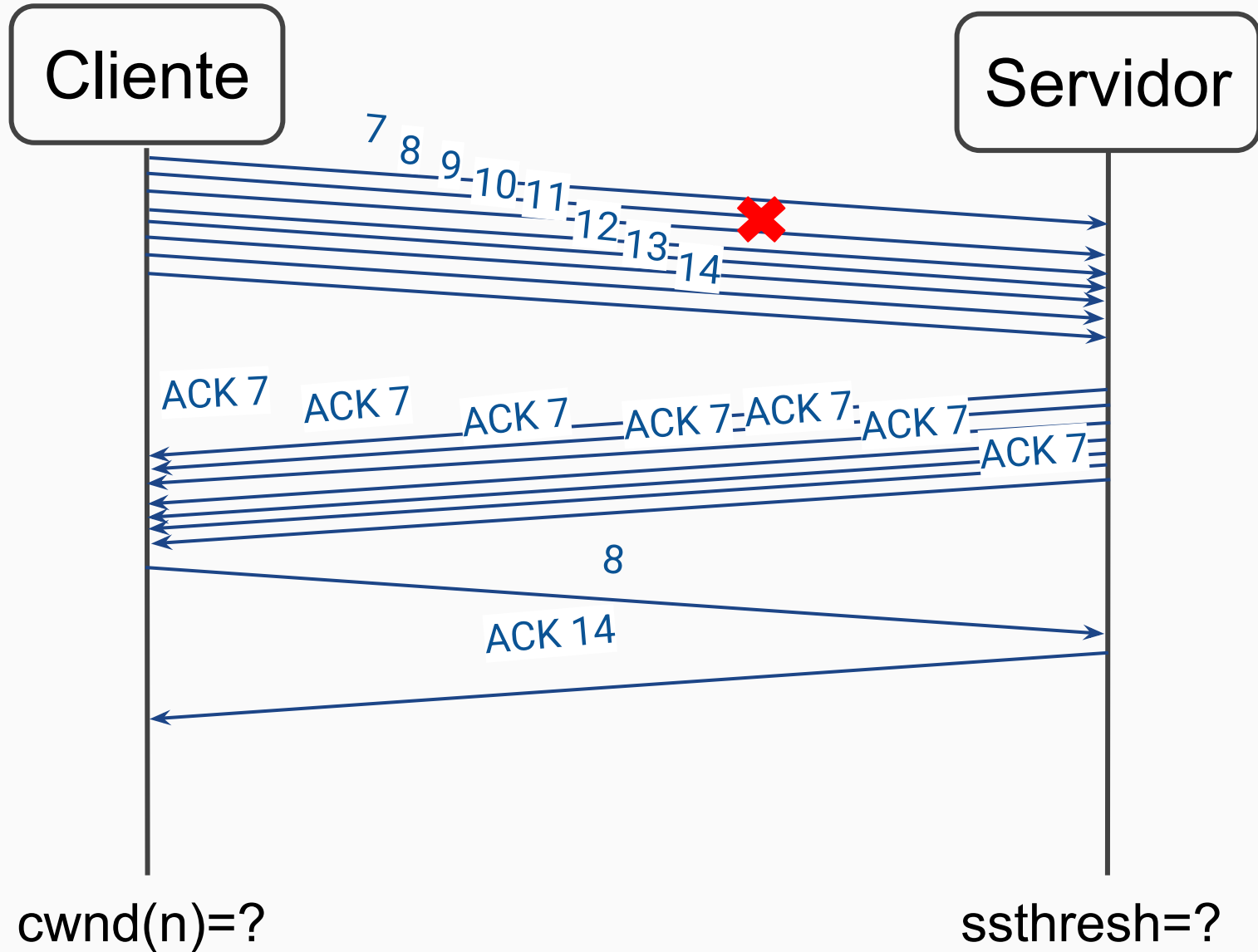
Slow start



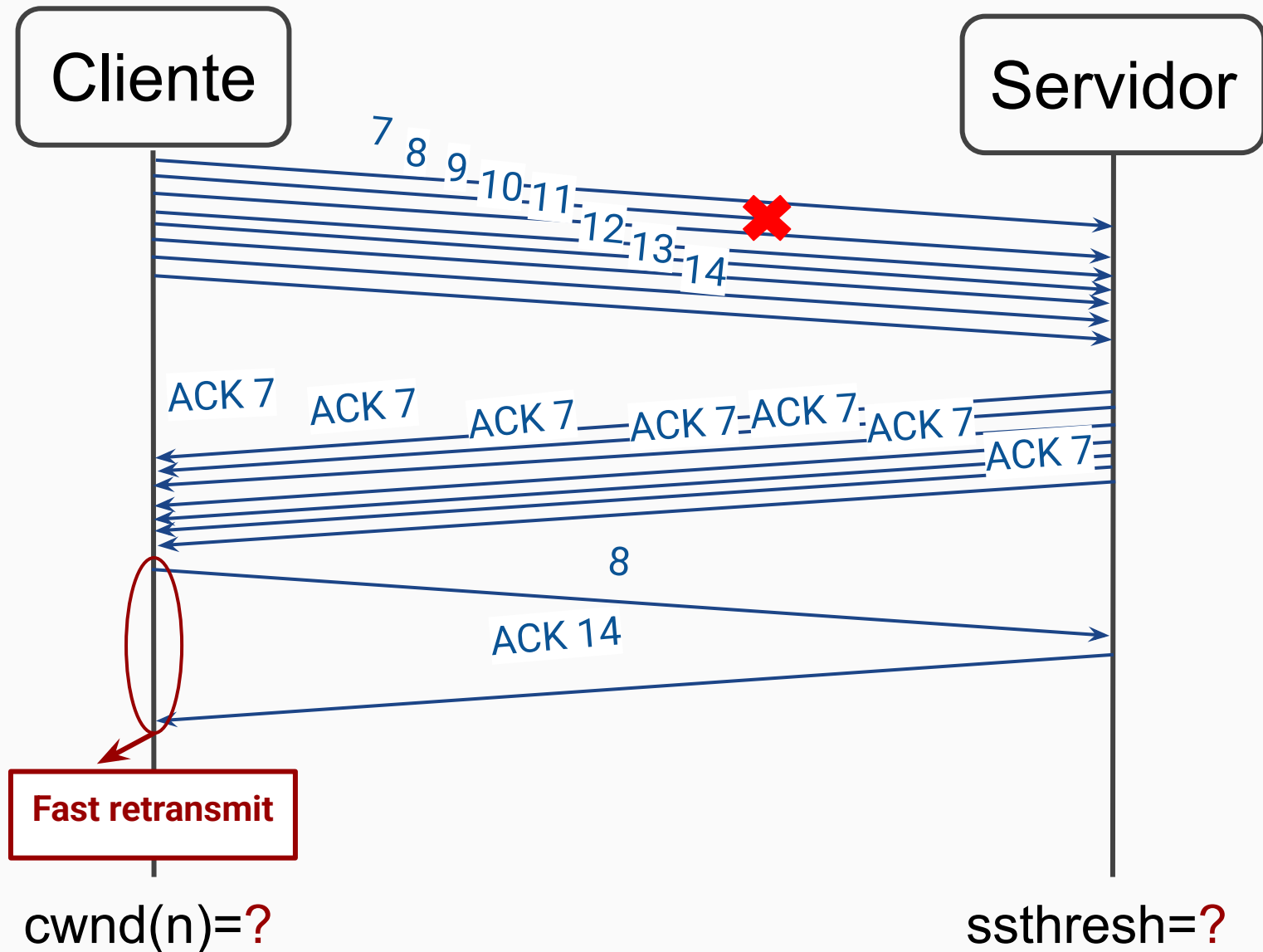
Slow start



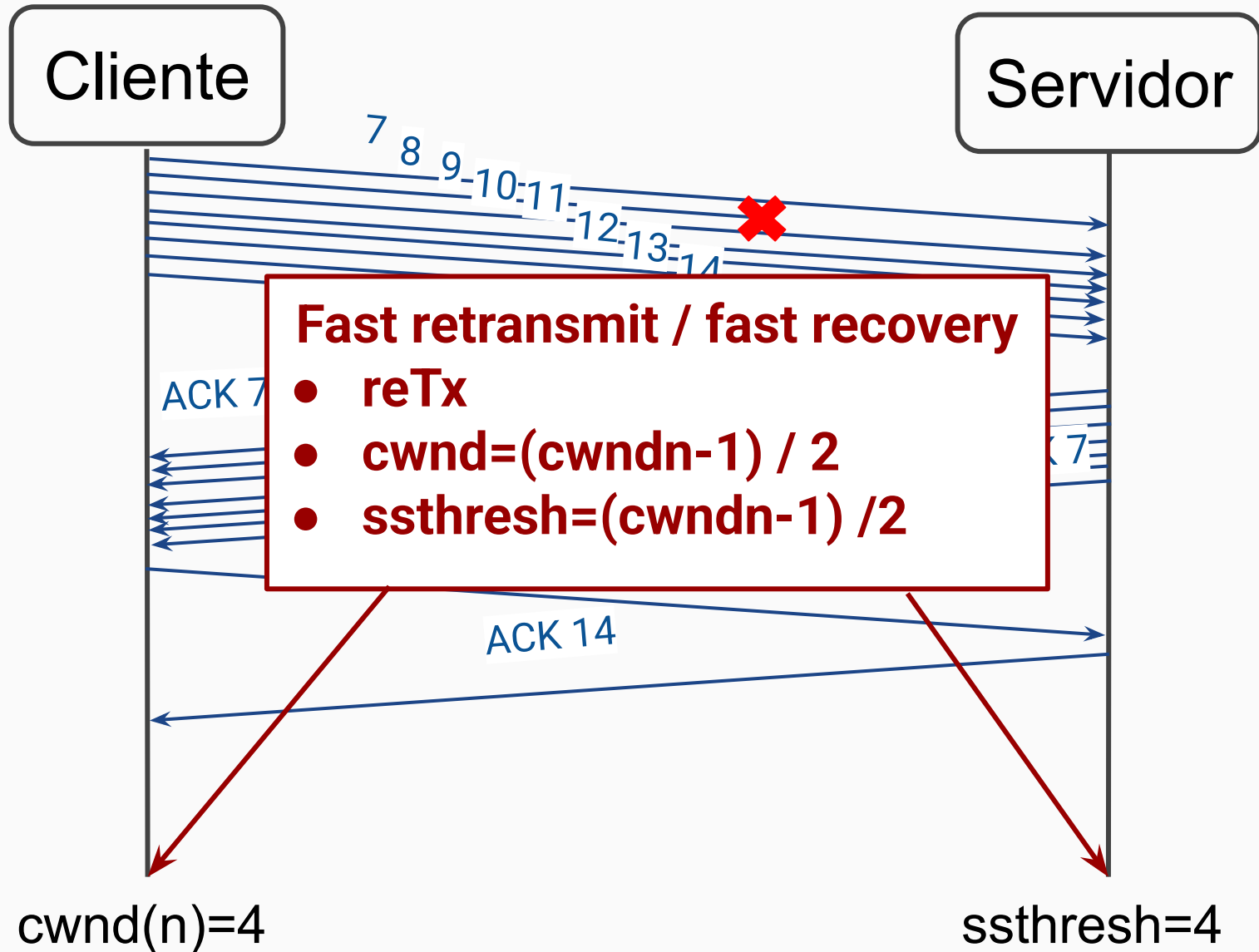
Slow start



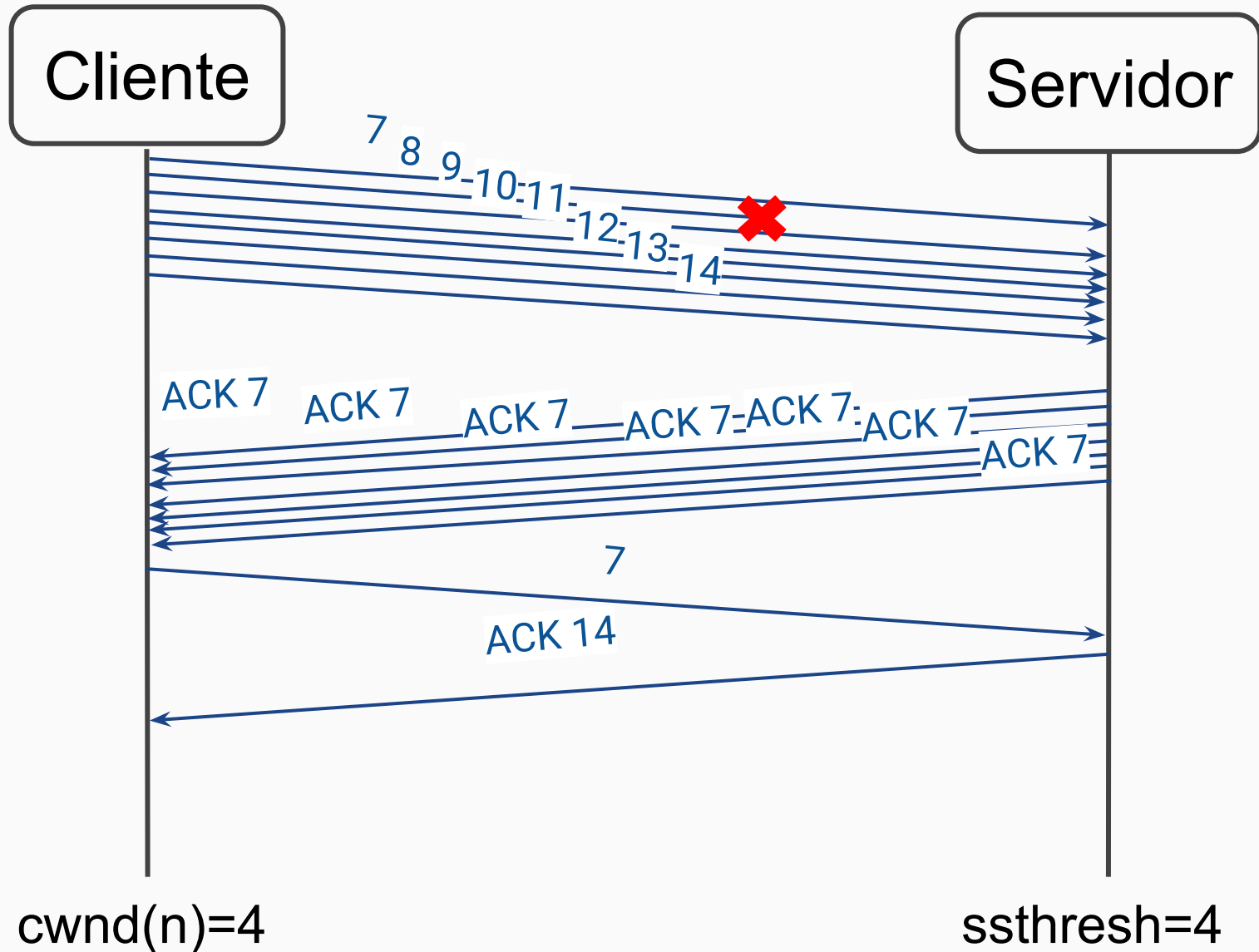
Fast retransmit



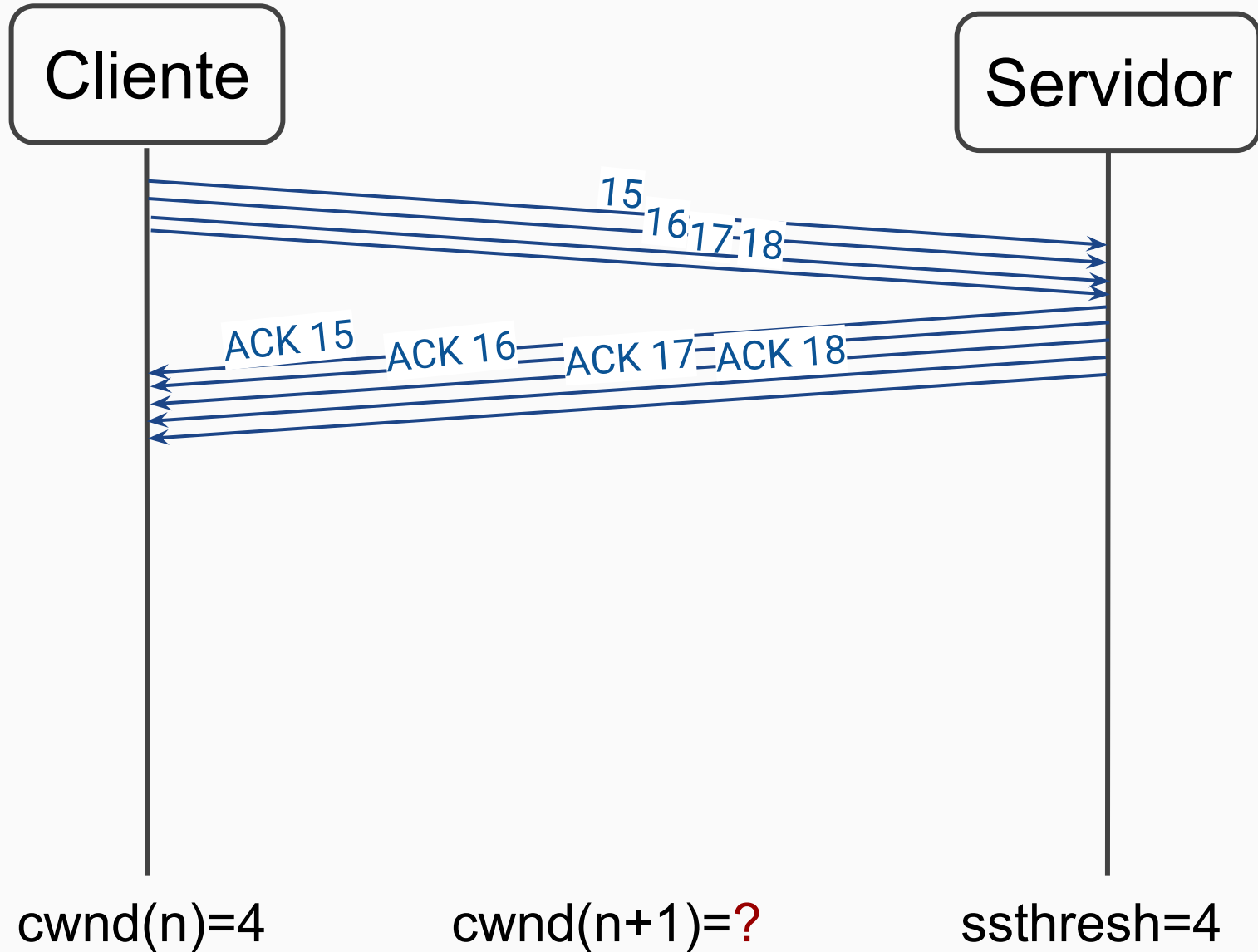
Fast retransmit



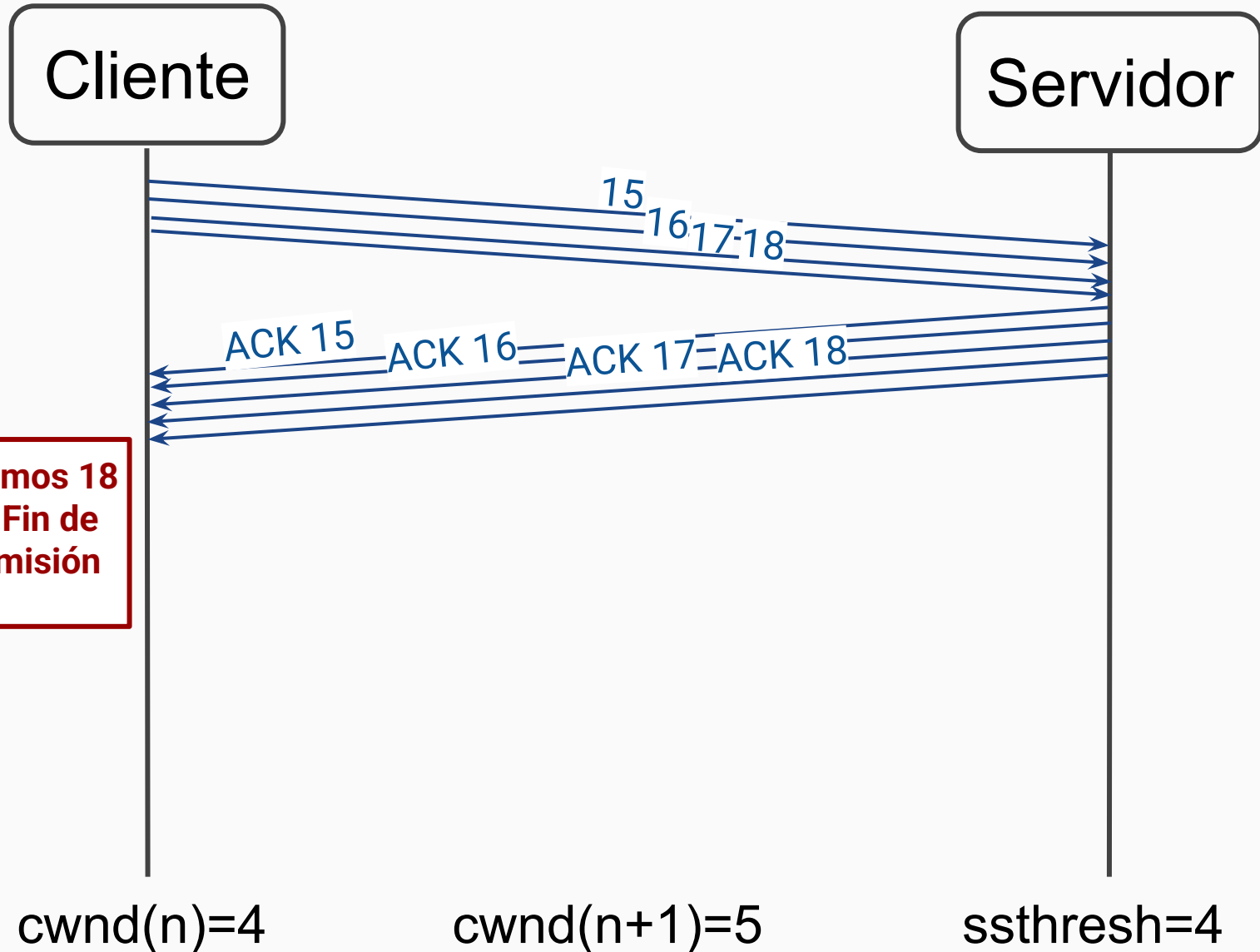
Fast retransmit



Fast retransmit



Fast retransmit



TCP : control de congestión

Otros algoritmos

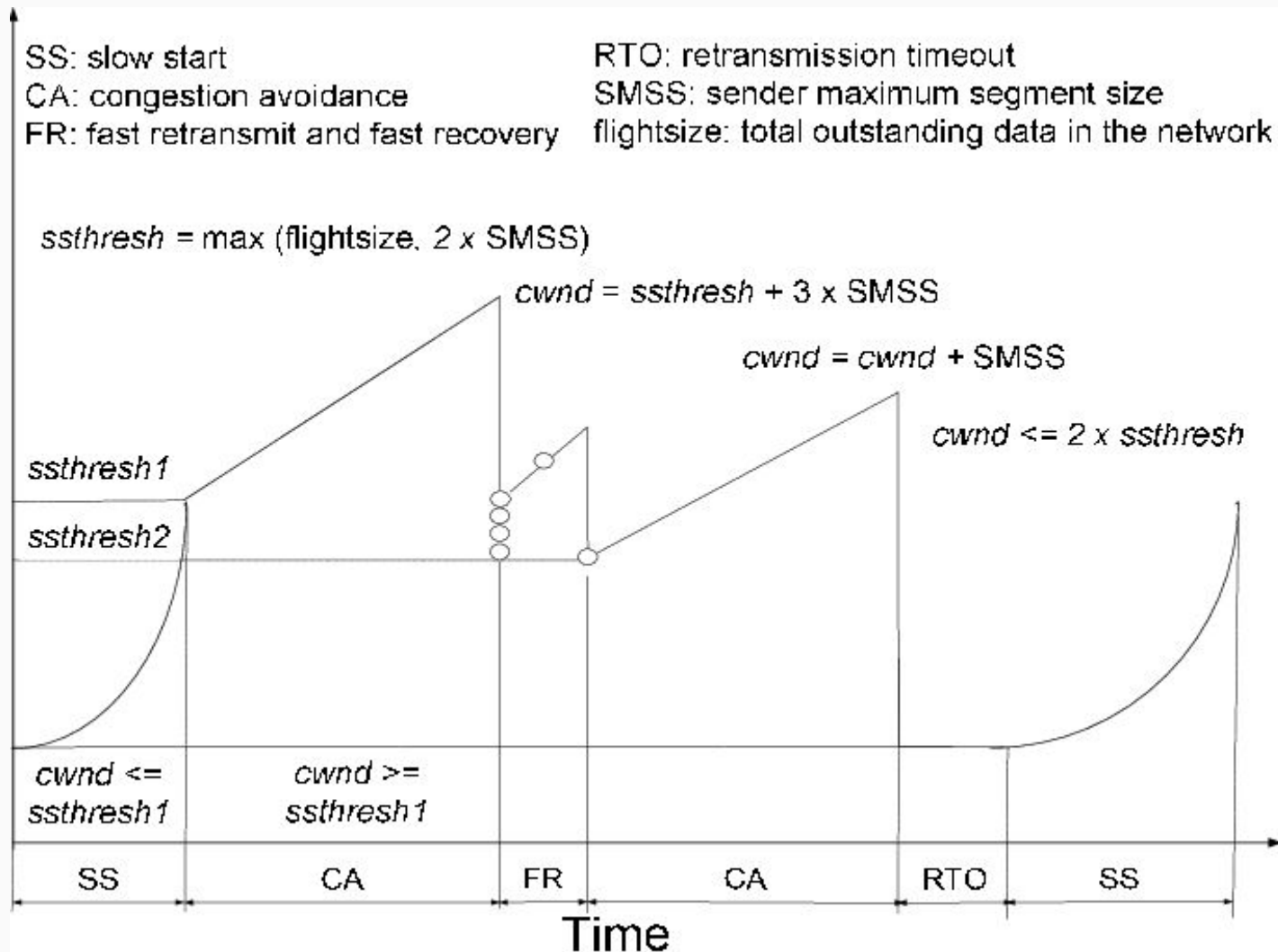
- TCP Tahoe y Reno son los más simples a nivel didáctico
- En la práctica ya no se utilizan. Hoy en día se usan algoritmos más modernos y complicados

TCP : control de congestión

Otros algoritmos

- Algunos ejemplos
 - TCP Vegas
 - TCP New Reno (viene por default en FreeBSD)
 - TCP CUBIC (viene por default en Linux)

TCP : control de congestión



¿Preguntas?

Referencias

- TCP: [RFC 793](#)
- TCP - Control de Congestion: [RFC 5681](#)